QSC Native Application Binary Interface - QSCABI

Revision 0.10.0-dev (public header baseline), November 8, 2026

Quantum Resistant Cryptographic Solutions Corporation

An engineering-level specification of the QSCABI shared-library interface, its language-binding contract, cryptographic and protocol services, build integration, exported functions, ownership model, validation expectations, and source reference material.

The QSCABI implementation and associated materials are commercial QRCS assets subject to the QRCS Commercial Application Software License (QRCS-CASL). The controlling license is supplied separately with authorized distributions.

Contents

1 Foreword 3
2 Introduction and Baseline 3
3 References 4
4 Terminology, Types and Status 5
5 ABI Architecture and Compatibility 8
6 Building, Linking and Packaging 9
7 Core Runtime and Memory 10
8 Encoding and Serialization 11
9 Hash and XOF Services 12
10 MAC and Key Derivation 12
11 Symmetric Ciphers and AEAD 13
12 Entropy and Platform Services 14
13 Post-Quantum Cryptography 14
14 Classical Asymmetric Cryptography 15
15 Explicit Algorithm Selection 15
16 Incremental Streaming Interfaces 16
17 Hardware Security Provider Bridge 16
18 X.509 Certificate Services 17
19 TLS 1.3 Connection and Server Services 18

20 Language Binding Design 19
21 Implementation Examples 19
22 Validation and Release Qualification 20
23 Complete Public API Reference 22
Annex A Public ABI Data Types and Constants 94
Annex B Source and Test Index 95
Annex C Implementation Checklist 96

1: Foreword

This document defines the implemented QSCABI native boundary for engineers creating or maintaining C, C++, C#, Java, Python and Rust clients. It describes the declarations contained in QSCABI(4).zip, including the 329 unique QSCABI_API exported function declarations in twenty public service headers. Each function is inventoried in Section 23 using its actual signature and source-header Doxygen notation.

QSCABI exposes a stable C calling interface over the external QSC cryptographic library. Its purpose is to insulate consumers from QSC native structure layouts, C++ name mangling, compiler alignment rules, cryptographic compile-time parameter selections and native memory ownership differences. The wrapper is not a second cryptographic implementation; algorithms and most native operations remain within QSC.

All examples and release assertions must be checked against the exact QSC binary and target platform chosen by the implementor. A public symbol present in a header does not demonstrate availability of its underlying hardware, entropy source, filesystem credential, curve parameter selection, or peer interoperability.

2: Introduction and Baseline

2.1 Purpose and supported environments

The public headers use portable C scalar and pointer types, size_t byte lengths, fixed-width integers, qscabi_status values, and opaque pointers for native state. A consumer includes qscabi.h (which collects the service headers), links or loads the QSCABI shared binary, and never imports QSC internal headers into its foreign-language binding layer. QSCABI therefore provides a durable boundary for managed applications and stable native integrations.

An application still owns its memory, cryptographic policies, private key protection, certificate trust decisions, network identity, and algorithm selection. QSCABI uses explicit lengths and output capacities so an FFI can validate memory safety before passing data to native code.

2.2 Source snapshot and document precedence

Item	Source-grounded value / status
Primary source archive	QSCABI(4).zip, supplied November 8, 2026
Public numeric ABI	0.10.0; QSCABI_VERSION_STRING is "0.10.0-dev" in qscabi_common.h

Item	Source-grounded value / status
Exported interface count	329 distinct functions declared with QSCABI_API across 20 public service headers
Version documentation discrepancy	docs/STAGE11_DOCUMENTATION.md describes 0.11.0-stage11, but the actual uploaded qscabi_common.h remains 0.10.0-dev; public header controls this paper
External dependency	QSC FULL-profile native library; uploaded archive does not contain the external QSC sources
Build artifacts supplied	Visual Studio QSCABI and QSCABITest projects plus object files/logs; no complete root CMakeLists.txt or QSCABI.public-symbols.txt manifest in this archive
License	QRCS Commercial Application Software License (QRCS-CASL); some legacy test fixtures noted by the project as needing licensing reconciliation
Qualification	Project historical logs only; this document-generation session did not compile or execute QSCABI

Table 2.1 Source baseline and provenance

3: References

3.1 Controlling project documents

- QSCABI/qscabi.h — Umbrella public API header and included service headers.
- QSCABI/qscabi_common.h — Version macros, C linkage and export definitions.
- docs/ABI_REFERENCE.md — Human-readable, header-organized exported function list.
- docs/QSCABI_DESIGN.md — Module scope, exclusions, dependency boundaries and ABI design rules.
- docs/MEMORY_OWNERSHIP.md — Caller-owned data, output lengths, secure clearing and allocator boundaries.
- docs/HANDLE_MODEL.md — Opaque handle creation, destruction and borrowed-buffer constraints.
- docs/STATUS_CODES.md — Stable status enum classification and error translation.
- docs/BUILD_GUIDE.md — Declared backend modes, integration requirements and platform qualification notes.
- docs/LANGUAGE_BINDING_GUIDE.md — Managed runtime mapping and handle ownership recommendations.
- docs/STAGE10_IMPLEMENTATION.md — TLS integration and recorded intermittent full-suite PQC limitation.
- docs/STAGE11_DOCUMENTATION.md — Document catalog; inconsistent version metadata noted above.

- QSCABIExamples/*.c — Seven demonstrative native C examples.
- QSCABITest/*.c — Per-module test entry points and direct test invocation.

3.2 External standards and references

QSC cryptographic library — https://github.com/Steppenwolfe65/QSC

C language interoperability overview (ISO C) — https://www.iso.org/standard/82075.html

NIST FIPS 197 AES — https://doi.org/10.6028/NIST.FIPS.197-upd1

NIST FIPS 202 SHA-3 — https://doi.org/10.6028/NIST.FIPS.202

NIST SP 800-185 cSHAKE and KMAC — https://doi.org/10.6028/NIST.SP.800-185

NIST FIPS 203 ML-KEM — https://doi.org/10.6028/NIST.FIPS.203

NIST FIPS 204 ML-DSA — https://doi.org/10.6028/NIST.FIPS.204

NIST FIPS 205 SLH-DSA — https://doi.org/10.6028/NIST.FIPS.205

RFC 8446 TLS 1.3 — https://www.rfc-editor.org/rfc/rfc8446

RFC 5280 X.509 PKI — https://www.rfc-editor.org/rfc/rfc5280

RFC 5869 HKDF — https://www.rfc-editor.org/rfc/rfc5869

CMake documentation — https://cmake.org/documentation/

4: Terminology, Types and Status

4.1 Core terminology

Term	Meaning for QSCABI consumers
ABI	Stable native binary contract: symbol names, calling convention, parameter widths and opaque pointer semantics.
Native QSC	Implementation library retained outside the public QSCABI header boundary.
FFI	Foreign function interface from a language runtime to native C functions.
Handle	Opaque process-local QSCABI-owned object pointer; never copied, dereferenced or directly freed by caller.
Borrowed pointer	Caller or library retains ownership; validity bound to the documented call or object lifetime.
Output capacity	Number of caller-allocated writable bytes; distinct from actual written byte count.
KEM	Key encapsulation mechanism, which must be combined with independent peer authentication.

Term	Meaning for QSCABI consumers
AEAD	Authenticated encryption with associated data; payload release requires tag verification.
FULL profile	Complete native QSC feature set required by the published 329-function ABI.
ABI version negotiation	qscabi_abi_is_compatible and capability flags; neither certifies crypto nor hardware presence.

Table 4.1 Terminology

4.2 Stable status model

Value	Constant	Meaning
0	qscabi_status_success	Operation completed successfully.
1	qscabi_status_failure	Operation failed for a general or unspecified reason.
2	qscabi_status_invalid_argument	One or more arguments were invalid, including null pointers where data was required.
3	qscabi_status_invalid_size	One or more length or size values were invalid for the requested operation.
4	qscabi_status_buffer_too_small	Caller-owned output buffer was smaller than the required output size.
5	qscabi_status_allocation_failure	Native memory allocation failed.
6	qscabi_status_authentication_failure	Authentication failed, such as an AEAD tag or MAC verification failure.
7	qscabi_status_verification_failure	Verification failed, such as a signature, certificate, or policy check failure.
8	qscabi_status_platform_unavailable	Required platform service, CPU feature, or entropy provider is unavailable.
9	qscabi_status_unsupported	Requested operation, algorithm, parameter set, or platform path is unsupported.
10	qscabi_status_not_initialized	Object or subsystem has not been initialized.
11	qscabi_status_already_initialized	Object or subsystem has already been initialized.
12	qscabi_status_invalid_handle	Opaque QSCABI handle was null, invalid, or not owned by the expected object type.
13	qscabi_status_bad_state	Object is in a state that does not allow the requested operation.
14	qscabi_status_not_found	Requested item, extension, certificate field, or named object was not found.
15	qscabi_status_io_failure	Input/output operation failed.
16	qscabi_status_tls_failure	TLS operation failed and may have additional TLS-specific detail internally.
17	qscabi_status_x509_failure	X.509 operation failed and may have additional certificate-specific detail internally.
18	qscabi_status_closed	Handle, connection, stream, or listener has been closed.

Table 4.2 Public qscabi_status values

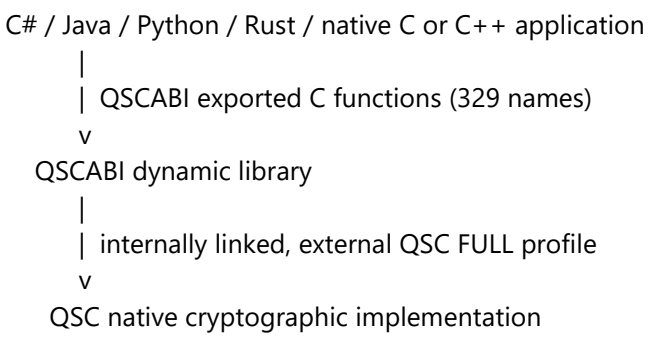
For functions returning qscabi_status, only qscabi_status_success authorizes consumption of their output. A size helper returning size_t, a boolean-like int, or a uint32_t compatibility predicate is not a qscabi_status and must be interpreted independently. qscabi_status_string returns an immutable static string; never free it. Log numeric status and operation context without exposing secrets.

4.3 C scalar widths and calling conventions

Header declarations use size_t for byte lengths; foreign-language bindings must represent size_t using the native process pointer width. uint32_t and uint64_t retain fixed width. Public enums cross C ABI boundaries with the representation selected by the compiler; this package assumes the supported native C ABI and tests the enum and integer model where possible. All C++ callers receive extern "C" linkage via qscabi_common.h. Windows DLL compilation selects QSCABI_EXPORTS, whereas consumers may select QSCABI_IMPORTS.

5: ABI Architecture and Compatibility

5.1 Dependency and ownership boundaries



All public QSCABI headers are written so language bindings need no QSC private header dependencies. Implementation sources compile against QSC internally; opaque state hides structure layout, alignment and platform configuration. QSCABI does not pass QSC heap allocations into a foreign allocator and does not require callers to release native buffers with a different CRT.

5.2 Binary stability and optional features

The uploaded headers expose 329 names and a major ABI value of zero, minor ten. Applications should check qscabi_abi_is_compatible(0U, 10U) before importing functions

added during the 0.10 series and query qscabi_capability_flags for optional sections. The symbols exist even when a hardware backend is disabled; an operation may then return qscabi_status_unsupported. Capability bits indicate compiled glue, not verified cryptographic security, usable instruction-set support or physical token presence.

5.3 Explicit exclusions

The source design expressly excludes a public SECRAND state lifecycle, generic raw sockets, arbitrary filesystem/collection/string tools, and QSC internal struct layouts. Language bindings should use their host environment for ordinary containers and file I/O. For TLS the public abstraction supplies contexts, connections, listeners and servers without passing the native socket handle structure across the interface.

6: Building, Linking and Packaging

6.1 Required native dependencies

QSCABI is designed to link a matching QSC FULL-profile library. Matching architecture, OS, ABI, C runtime, QSC compile-time parameters and supported processor instructions are essential. Build identifiers are diagnostic provenance only. A label such as FULL does not prove the supplied binary contains every required algorithm; the actual link and export audits remain mandatory.

6.2 Build configurations described by project documentation

Mode	How the source documentation describes it
TARGET	Use an already-defined QSC target, normally imported or built as part of the parent CMake project.
PREBUILT	Supply explicit matching QSC static library and include directory; confirm its FULL profile separately.
SOURCE	Use the QSC source inventory as an integration fallback, not a license to modify protected QSC files.
AUTO	Select a discovered target or permitted fallback according to existing build configuration.
MSVC project	Build QSC dependency first, QSCABI DLL second, then QSCABITest in Visual Studio.

Table 6.1 Documented build integration modes

The provided ZIP contains Visual Studio project files and a pre-existing Debug compilation trace. Its docs also refer to CMakeLists.txt, cmake/QSCABI.public-symbols.txt, package exports and installed targets; those particular build-system source files are not present in this uploaded archive. Consequently, the CMake commands

below are included as repository-documented procedures, not reproducible commands verified against this ZIP alone.

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release \
  -DQSCABI_QSC_BACKEND=PREBUILT \
  -DQSCABI_QSC_LIBRARY=/path/to/libqsc.a \
  -DQSCABI_QSC_INCLUDE_DIR=/path/to/QSC/Source/QSC \
  -DQSCABI_QSC_PROFILE=FULL
cmake --build build --parallel
ctest --test-dir build --output-on-failure
```

6.3 Export hygiene and native packaging

The project documentation describes an exact 329-name export manifest, Linux ELF symbol versioning/export controls, a macOS exported-symbol list, and explicit QSCABI_API dllexport rules on Windows. For final release, audit the actual shared library exports against declarations and ensure qsc_* native QSC functions do not leak. This archive provides no manifest file or compiled QSCABI DLL that could independently substantiate the exact exported set.

An installed-package consumer should link only QSCABI::QSCABI (where the CMake package is supplied) and load the correct native shared library for its platform. The loader search path, DLL signing, dependencies, Linux SONAME and RPATH, and ABI/architecture alignment are release packaging concerns, not cryptographic algorithm details.

7: Core Runtime and Memory

7.1 Version negotiation and capabilities

```
if (qscabi_abi_is_compatible(0U, 10U) != 1U)
{
    /* Refuse unsupported ABI before importing optional symbols. */
}
uint64_t features = qscabi_capability_flags();
const char* version = qscabi_version_string();
```

The library-owned strings from qscabi_version_string, qscabi_qsc_version_string, qscabi_build_backend, qscabi_build_compiler, qscabi_build_platform and qscabi_qsc_build_profile must not be modified or freed. qscabi_enabled_features() and capability flags are descriptive metadata rather than runtime attestation. qscabi_capability_flags uses a fixed uint64_t, while qscabi_abi_is_compatible returns a uint32_t predicate.

7.2 Caller buffers and clearing

Normal input/output memory remains with the caller. A nonzero length implies a valid pointer. QSCABI does not return allocated application buffers that must be released across the shared-library allocator boundary. On successful variable-size operations, the written count states how many bytes were produced; capacity alone is not the output length.

qscabi_memory_clear wipes a caller-owned byte range through the native memory utility. qscabi_memory_are_equal is a byte-equality predicate intended for constant-time comparison when supported by the underlying QSC implementation. Dispose caller-owned private key, seed, shared secret, MAC key, derived key and decrypted data buffers after use, including copies created by the language runtime.

7.3 Failure handling

A binding should convert non-success qscabi_status to a structured error, retain a numeric code and functional context for diagnostics, and leave result objects unusable unless all required output lengths and authentication checks succeeded. A buffer-too-small status does not authorize reading partially filled bytes. Invalid-handle and bad-state errors signal incorrect lifecycle management rather than a transient crypto error.

8: Encoding and Serialization

8.1 Hex, Base64 and PEM

qscabi_encoding.h provides size helpers and encode/decode functions with explicit capacities and actual byte counts. These functions operate on bytes; the language binding is responsible for character encoding, length units and NUL termination where a text form is required. Do not use a C string-length function on arbitrary binary outputs.

```
size_t encoded_size = qscabi_hex_encoded_size(binary_length);
/* Allocate encoded_size plus any required NUL terminator. */
qscabi_hex_encode(text, text_capacity, &chars_written,
                  binary, binary_length);
qscabi_hex_decoded_size(&decoded_size, text, chars_written);
```

A successful PEM decoding operation is only successful syntax handling; it is not certificate trust validation. The host runtime should select a single canonical serialization format and reject malformed, truncated or unexpectedly large encodings before passing material to an X.509 or cryptographic object.

9: Hash and XOF Services

9.1 SHA-2 and SHA-3

The one-shot hash module supplies digest-size helpers and QSC-backed SHA-2/SHA-3 computation. Query digest length and allocate a caller-owned buffer; pass exact input byte length; check the resulting qscabi_status. Hashes provide collision-resistant digests but cannot independently authenticate a sender or grant access.

```
size_t required = qscabi_sha2_256_digest_size();
qscabi_status result = qscabi_sha2_256_compute(
    digest, required, message, message_length);
/* Consume digest only if result == qscabi_status_success. */
```

9.2 SHAKE extendable output

qscabi_xof.h exposes SHAKE output length as a caller-selected number of bytes. A SHAKE output is not an HMAC or key-derivation protocol by itself; application-level domain separation must be designed deliberately. For streaming XOFs, absorb input, finalize once, then squeeze output incrementally under a single owned handle.

10: MAC and Key Derivation

10.1 MAC families

QSCABI includes HMAC-SHA2, KMAC, KMAC-XOF, Poly1305, QMAC and PBMAC1 interfaces according to its qscabi_mac.h declarations. Message authentication requires a secret key and an agreed input byte sequence. Use size helpers, explicitly selected outputs and suitable domain-separation strings. Do not interchange KMAC and HMAC tags merely because their lengths match.

10.2 Key derivation

The qscabi_kdf.h interface provides HKDF and PBMAC1 derivation services. Salt, input key material, info/personalization, iteration policy and output length serve different roles and should be represented as distinct binding parameters. Avoid sharing derived keys across protocols, and clear temporary key material promptly. A password-derived key has a separate policy requirement from an already high-entropy KEM or ECDH shared secret.

10.3 Authentication and comparison

Before accepting a protected message, recompute or verify the authenticator using the protocol-defined key, personalization and encoded fields. A successful digest computation is not a successful authentication check. For application-side comparison use qscabi_memory_are_equal only for exact equal-length secrets or tags and reject mismatched lengths before the comparison.

11: Symmetric Ciphers and AEAD

11.1 Algorithm families and security model

The qscabi_symmetric.h header exposes AES block/CTR/GCM, ChaCha, ChaCha20-Poly1305, CSX and RCS functionality, with size helpers and status-bearing one-shot operations. CTR and raw ChaCha protect confidentiality only; authenticated modes protect integrity when configured with a fresh nonce and validated tag. Repeated nonce use under one key is a serious failure.

11.2 Output capacity and overlap

The public ownership guide specifies out-of-place authenticated encryption. The ciphertext and plaintext input ranges must not overlap caller outputs for AEAD. Failed authentication must not expose tentative plaintext, and the ABI reports zero written bytes on authentication errors where documented. Managed wrappers should allocate a separate output buffer, verify status, and copy into an application-owned result only on success.

11.3 CTR stream handling

AES CTR modes may be applied to byte lengths, but counter evolution and subsequent key/nonce use are part of the protocol contract. Incremental qscabi_ctr_state handles require create-transform-destroy ownership, one-thread-at-a-time access, and no implied authentication. Any transport accepting CTR ciphertext must add an independent message authenticator.

12: Entropy and Platform Services

12.1 Provider-level random bytes

The entropy surface consists of ACP, CSP, RDP and CJP provider availability/generation calls. No SECRAND native state or reseed lifecycle is exported. Availability is platform-

dependent and should be checked or handled via returned status. Random output is not a test vector; deterministic comparisons are unsuitable for entropy validation.

12.2 Security and failure policy

A platform-unavailable status must not silently fall back to predictable test bytes. The application should document whether entropy-provider unavailability prevents key generation, handshake continuation or another security-sensitive operation. A language binding must not substitute its ordinary host PRNG for QSC cryptographic randomness without a separately reviewed design.

13: Post-Quantum Cryptography

13.1 KEM and signature families

The qscabi_pqc.h API includes ML-KEM/Kyber, HQC, Classic McEliece, ML-DSA/Dilithium, Falcon and SLH-DSA/SPHINCS+ wrappers. KEM key generation, encapsulation and decapsulation use exact byte-length requirements; signatures use their own detached or attached-message formats as specified by each public function. Native parameter sets available through the default facade depend on the compiled QSC library.

13.2 KEM acceptance and authentication

A successful KEM roundtrip produces a shared secret but does not authenticate the encapsulating party. Implicit rejection may deliberately produce plausible secret bytes for a malformed encapsulation, so caller authentication and transcript binding remain mandatory. Do not infer protocol success solely from nonzero shared secret bytes.

13.3 Signature verification and output

Use the exact signature-size helper and verify the expected message representation. An attached signature differs from a detached signature in both buffer sizing and the data that verification returns. The tests include tamper cases and algorithm-specific negative outcomes. A cryptographically valid signature does not by itself grant application authorization.

14: Classical Asymmetric Cryptography

14.1 ECDH and EDDH

The classical asymmetric module exposes key-size helpers, key generation, seeded generation, public-from-private operations and shared secret agreement for selected native ECDH and EDDH algorithms. Check each returned size rather than hard-coding a curve choice in a managed wrapper. Derive application keys from any shared secret using the agreed KDF and transcript binding.

14.2 ECDSA and EDDSA

The signature APIs include attached-message size calculation and sign/verify operations. A foreign-language wrapper must respect whether a function returns the original message through an output parameter or returns a detached signature. Convert qscabi_status_verification_failure into a verification outcome, not an I/O or allocation exception.

15: Explicit Algorithm Selection

15.1 Algorithm selectors and size queries

qscabi_explicit_kem.h, qscabi_explicit_curve.h, qscabi_explicit_signature.h and qscabi_explicit_slhdsa.h add stable selector enums and get_sizes outputs. These APIs let a language binding name supported parameters explicitly rather than depending exclusively on a QSC compile-time default. A selector must be validated against the native implementation; unsupported remains an expected status.

15.2 SLH-DSA domain separation and prehash

The SLH-DSA extension exposes SHAKE variants, optional prehash policy, randomized/deterministic signing selection, and a context string limited by the current header to 255 bytes. Key and signature buffer sizes must match the selected algorithm. The prehash policy is constrained by the selected native security level; code must reject unsupported combinations, not downgrade.

16: Incremental Streaming Interfaces

16.1 Opaque state-machine contracts

Handle class	Required sequence	Important restriction
qscabi_digest_state	create → update* → finalize → destroy	Digest finalization occurs once; reject repeated finalize.
qscabi_xof_state	create → absorb* → finalize → squeeze* → destroy	Absorb only before finalize; squeeze may repeat.
qscabi_stream_mac_state	create → update* → finalize → destroy	Check selected algorithm, key and exact output size.
qscabi_ctr_state	create → transform* → destroy	CTR is unauthenticated; do not share native state between threads.

Table 16.1 Streaming lifecycle contracts

Opaque state pointers are process-local and must be bound to one owner in the host runtime. The current headers describe a single-thread-owned state model; protect or prohibit concurrent access explicitly. Release state via the matching destroy function, including after failed updates and finalization. Do not copy state bytes, invent object layouts, or pass state handles after destruction.

17: Hardware Security Provider Bridge

17.1 Callback contracts and custody

The hardware layer exposes provider and key handles bound to application-supplied callback operations and an application-managed backend context. qscabi_hw_provider_open copies callback-table descriptions but borrows the backend context, which must remain alive until close. The bridge may support native key generation, sign/agree, public-key extraction, attestation and provider randomness when an enabled backend supplies these operations.

17.2 Not a hardware-presence attestation

qscabi_hw_is_enabled and the hardware capability bit indicate that the bridge was compiled with relevant QSC hardware support, not that a physical token is connected. Devices, provider libraries, key non-exportability, firmware provenance, attestation trust and lifecycle failure modes require independent device-specific tests. A security policy that mandates hardware keys must fail if no qualifying backend is available.

17.3 Lifetime and thread safety

Release opened key handles before invalidating or closing a provider. Callback reentrancy, cancellation, device disconnects and stale-handle access must be handled by the application's callback implementation. A managed wrapper should store callback delegates or trampolines in an owning object whose lifetime exceeds every native operation that might use them.

18: X.509 Certificate Services

18.1 Certificate, chain and trust-store handles

The X.509 bridge provides opaque certificate, chain and store handle types. Loading certificate DER/PEM into owned QSCABI objects copies backing bytes before parsing; this avoids dangling native pointers when QSC parsing retains borrowed slices. Reload replaces the previous backing store after native state is cleared. The caller owns encoded source arrays only for the call itself.

18.2 Verification and hostname policy

A parsed certificate or a successful host-string extraction does not establish trust. Create a trust store from independently authorized anchors, supply the presented peer certificates, validate the chain with a defined profile, and separately check expected DNS identity and usage constraints. Treat every non-success result and unknown revocation state according to an explicit application policy.

18.3 CSR issuance and CRL operations

The expanded 0.10 X.509 surface includes P-256 CSR creation, DNS extensions, DER signing, CSR load/verify, restricted TLS-server certificate issuance, CRL-builder lifecycle, revocation entries and signed CRL verification. Signing functions accept callback-based ECDSA signing where documented; an implementor must maintain the callback pointer, signing key custody and the correct DER signature format throughout each synchronous operation.

19: TLS 1.3 Connection and Server Services

19.1 Opaque TLS object hierarchy

```
qscabi_tls_context_create
  -> set policy / socket options / ALPN / SNI / auth / tickets
  -> load certificate and trust file paths
  -> create connection (client) or listener/server (service)
  -> negotiate authenticated TLS 1.3
  -> send/receive or framed send/receive
  -> shutdown / close / destroy
```

QSCABI provides TLS context, connection, listener and server handles. Configuration is not a completed handshake; check the connection and handshake completion methods and peer status before processing protected application messages. The ABI intentionally hides raw QSC socket context structure and does not expose a generic raw-socket transport API.

19.2 Credential loaders and compatibility stubs

The two original in-memory PEM trust/identity entry points are retained as exported symbols but return qscabi_status_unsupported on otherwise valid input in this implementation. Use the supported additive file-based trust-anchor, bundle, CRL, server/client identity and SNI identity loaders. These require filesystem paths with lifetime and permissions managed by the application.

19.3 Policy preferences and framing

Cipher suites, named groups and signature IDs use native TLS 1.3 identifiers with element counts; unsupported or duplicate entries are rejected as described by the source. This does not guarantee negotiation with every peer. ALPN, client authentication, SNI requirements, ticket settings and socket options must be configured before starting a connection or listener.

Send/receive and framed send/receive return status and byte counts. Higher-layer framing, partial I/O, transport interruption and cancellation remain client obligations. Use shutdown and destroy functions in the correct order and do not share a mutable connection handle across host threads without explicit synchronization.

20: Language Binding Design

20.1 Shared FFI rules

Language	Type representation	Ownership / safety
C/C++	include qscabi.h; extern C; size_t native width	Check every status and use library create/destroy for handles.
C#	DllImport / Cdecl, nuint/UIntPtr for size_t	SafeHandle and deterministic disposal; pin Span memory across native calls.
Python	ctypes.c_size_t, c_uint8, explicit argtypes/restype	ctypes buffers and context managers; avoid __del__ as sole cleanup.
Java/JNI	jlong checked size conversions, direct buffers or bounded copies	Own native handles in closeable objects; pin/copy arrays with JNI rules.
Rust	extern C, usize matching target ABI	unsafe FFI internals, safe typed façade, Drop and secret zeroization.

Table 20.1 Language interoperability contracts

20.2 Capability gating and exceptional results

Every wrapper should expose its loaded ABI major/minor and feature bits, reject incompatible binaries before optional symbols are loaded, and represent operation results with typed errors. Never confuse a status enum with a C boolean or size_t. Exception messages should use static status descriptions but must not include plaintext, long-term private keys, password inputs or secret buffers.

20.3 Native handle wrappers

Use deterministic resource management: SafeHandle (C#), Closeable/AutoCloseable (Java), close/context manager (Python) or Drop (Rust). A managed reference to an opaque pointer is not permission to duplicate ownership or close the same pointer twice. Design thread-affinity rules for streaming, TLS and callback-based hardware operations before publishing safe high-level wrappers.

21: Implementation Examples

21.1 Examples supplied by the source project

C source	Implementor learning objective
abi_capabilities_example.c	Version, platform/compiler/profile strings and major/minor feature gating.
hash_sha256_example.c	SHA2-256 known-answer digest, success status and temporary buffer clearing.

C source	Implementor learning objective
hex_roundtrip_example.c	Hex encoded/decoded size queries, actual written counts and byte equality.
hmac_sha256_example.c	HMAC-SHA2-256 generation and comparison with key cleanup.
mlkem_roundtrip_example.c	ML-KEM size queries, key generation, encapsulation/decapsulation, shared-secret comparison.
tls_configuration_example.c	TLS policy, socket/ticket settings, suite/group/signature preferences and trust file.
x509_csr_prepare_example.c	P-256 size query, key generation, CSR creation, subject/DNS attributes and cleanup.

Table 21.1 Uploaded native examples

The examples demonstrate selected APIs, not complete language wrapper implementations. The TLS configuration example does not demonstrate an independent live peer handshake. The CSR preparation example does not issue a production certificate. Replace demonstration keys, trust paths and policy with reviewed inputs before deployment.

21.2 Representative call hierarchies

```
Version:  qscabi_abi_is_compatible -> qscabi_capability_flags
SHA-256:  qscabi_sha2_256_digest_size -> qscabi_sha2_256_compute
ML-KEM:   key_size helpers -> generate_keypair -> encapsulate -> decapsulate
X.509:    create -> load -> query / validate -> destroy
TLS:      context_create -> configure -> load files -> connect/accept
        -> send/receive -> shutdown -> destroy
```

22: Validation and Release Qualification

22.1 Test runner and source evidence

The QSCABITest project contains directly invoked named test modules for core, encoding, hash, XOF, MAC, KDF, symmetric, entropy, PQC, classical asymmetric, X.509, TLS and later stage extensions. The project testing guide documents non-interactive -test, -run all and default full-suite operation; only -interactive should read commands and optionally block for terminal close. Every test should report a distinct PASS/FAIL status and support repeatable exit codes for CI.

```
QSCABITest -test core
QSCABITest -test symmetric
QSCABITest -test pqc
QSCABITest -test tls
QSCABITest -run all
```

22.2 Scope and open qualification issues

The supplied Stage 10 individual module logs report success for TLS, asymmetric, PQC and X.509 sections. The captured Stage 10 full-suite attempt stops during the inherited Falcon section and explicitly does not certify the full suite. A Stage 5 narrative reports 21/21 Linux PREBUILT CTest checks against a separate earlier build tree. These are historical source-package records; this specification task performed no compilation or runtime validation and must not be treated as a current release PASS.

The user archive omits the referenced CMake export manifests and complete CMake project root, and it contains no linked QSC dependency, Linux/macOS/ARM64 builds or full final release packages. Native Windows, macOS, ARM64, managed-language runtime smoke, live TLS two-host interoperability and physical hardware-provider testing require separate release evidence.

22.3 Release gate matrix

Gate	Minimum acceptance evidence
Header and symbol contract	329 public function declarations checked against linked exports and authoritative manifest, with zero unintended qsc_* exports.
Binary interoperability	Windows/MSVC, Linux/GCC and macOS/Clang native target builds with documented QSC FULL configuration.
Crypto behavior	Known-answer, roundtrip, negative length/overlap/tag/signature tests for each published family.
Resource ownership	Handle create/destroy, null destruction, repeated finalize, use-after-close prevention and memory clearing.
Managed language binding	C#/Python/Java/Rust loading, size_t and callback marshalling, correct exception conversion and disposal.
Transport validation	Real TLS peers, certificates, hostname and revocation failures, ALPN and client-auth scenarios.
Physical hardware	Manufacturer/firmware/provider, custody and non-exportability, invalidation, device disconnect and attestation.
Distribution	QRCS-CASL licensing consistency, package manifest, signatures, binary hashes and installable consumer test.

Table 22.1 Release acceptance criteria

23: Complete Public API Reference

The following reference covers every QSCABI_API declaration found in the uploaded public service headers, with exact public prototypes and source-header Doxygen descriptions, parameter annotations, status/return notes, and significant safety remarks. Documentation is generated from the current uploaded headers rather than superseded earlier source packages; executable behavior still requires testing of the linked QSC library and wrapper implementation.

23.1 qscabi_asymmetric.h — 36 functions

Compile-time selected ECDH, EDDH, ECDSA and EDDSA operations and sizing.

qscabi_ecdh_public_key_size

size_t qscabi_ecdh_public_key_size(void);

Description: Gets the ECDH public-key size.

Return: The required ECDH public-key size in bytes.

qscabi_ecdh_private_key_size

size_t qscabi_ecdh_private_key_size(void);

Description: Gets the ECDH private-key size.

Return: The required ECDH private-key size in bytes.

qscabi_ecdh_shared_secret_size

size_t qscabi_ecdh_shared_secret_size(void);

Description: Gets the ECDH shared-secret size.

Return: The ECDH shared-secret size in bytes.

qscabi_ecdh_seed_size

size_t qscabi_ecdh_seed_size(void);

Description: Gets the ECDH deterministic seed size.

Return: The seed size in bytes required by qscabi_ecdh_generate_seeded_keypair.

qscabi_ecdh_generate_keypair

qscabi_status qscabi_ecdh_generate_keypair(uint8_t* publickey, size_t publickey_length, uint8_t* privatekey, size_t privatekey_length);

Description: Generates an ECDH key pair.

Details: The generated public and private keys are written to caller-owned output buffers. The buffers must be sized using qscabi_ecdh_public_key_size and qscabi_ecdh_private_key_size.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public-key buffer in bytes.; privatekey: The caller-owned output buffer that receives the private key.; privatekey_length: The size of the private-key buffer in bytes.

Return: Returns qscabi_status_success on success, otherwise an error status.

qscabi_ecdh_generate_seeded_keypair

qscabi_status qscabi_ecdh_generate_seeded_keypair(uint8_t* publickey, size_t publickey_length, uint8_t* privatekey, size_t privatekey_length, const uint8_t* seed, size_t seed_length);

Description: Generates a deterministic ECDH key pair from a seed.

Details: The seed must be exactly qscabi_ecdh_seed_size bytes. The generated key pair is written to caller-owned buffers.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public-key buffer in bytes.; privatekey: The caller-owned output buffer that receives the private key.; privatekey_length: The size of the private-key buffer in bytes.; seed: The caller-owned input seed.; seed_length: The size of the seed in bytes.

Return: Returns qscabi_status_success on success, otherwise an error status.

qscabi_ecdh_public_from_private

qscabi_status qscabi_ecdh_public_from_private(uint8_t* publickey, size_t publickey_length, const uint8_t* privatekey, size_t privatekey_length);

Description: Derives an ECDH public key from a private key.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public-key buffer in bytes.; privatekey: The caller-owned input private key.; privatekey_length: The size of the private key in bytes.

Return: Returns qscabi_status_success on success, otherwise an error status.

qscabi_ecdh_key_exchange

qscabi_status qscabi_ecdh_key_exchange(uint8_t* secret, size_t secret_length, const uint8_t* privatekey, size_t privatekey_length, const uint8_t* publickey, size_t publickey_length);

Description: Computes an ECDH shared secret.

Details: The secret buffer must be sized using qscabi_ecdh_shared_secret_size. The private key is the local private key and the public key is the peer public key.

Parameters: secret: The caller-owned output buffer that receives the shared secret.; secret_length: The size of the shared-secret buffer in bytes.; privatekey: The caller-owned input local private key.; privatekey_length: The size of the private key in bytes.; publickey: The caller-owned input peer public key.; publickey_length: The size of the public key in bytes.

Return: Returns qscabi_status_success on success, otherwise an error status.

qscabi_eddh_public_key_size

size_t qscabi_eddh_public_key_size(void);

Description: Gets the EDDH public-key size.

Return: The required EDDH public-key size in bytes.

qscabi_eddh_private_key_size

size_t qscabi_eddh_private_key_size(void);

Description: Gets the EDDH private-key size.

Return: The required EDDH private-key size in bytes.

qscabi_eddh_shared_secret_size

size_t qscabi_eddh_shared_secret_size(void);

Description: Gets the EDDH shared-secret size.

Return: The EDDH shared-secret size in bytes.

qscabi_eddh_seed_size

size_t qscabi_eddh_seed_size(void);

Description: Gets the EDDH deterministic seed size.

Return: The seed size in bytes required by qscabi_eddh_generate_seeded_keypair.

qscabi_eddh_generate_keypair

qscabi_status qscabi_eddh_generate_keypair(uint8_t* publickey, size_t publickey_length, uint8_t* privatekey, size_t privatekey_length);

Description: Generates an EDDH key pair.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public-key buffer in bytes.; privatekey: The caller-owned output buffer that receives the private key.; privatekey_length: The size of the private-key buffer in bytes.

Return: Returns qscabi_status_success on success, otherwise an error status.

qscabi_eddh_generate_seeded_keypair

qscabi_status qscabi_eddh_generate_seeded_keypair(uint8_t* publickey, size_t publickey_length, uint8_t* privatekey, size_t privatekey_length, const uint8_t* seed, size_t seed_length);

Description: Generates a deterministic EDDH key pair from a seed.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public-key buffer in bytes.; privatekey: The caller-owned output buffer that receives the private key.; privatekey_length: The size of the private-key buffer in bytes.; seed: The caller-owned input seed.; seed_length: The size of the seed in bytes.

Return: Returns qscabi_status_success on success, otherwise an error status.

qscabi_eddh_public_from_private

qscabi_status qscabi_eddh_public_from_private(uint8_t* publickey, size_t publickey_length, const uint8_t* privatekey, size_t privatekey_length);

Description: Derives an EDDH public key from a private key.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public-key buffer in bytes.; privatekey: The caller-owned input private key.; privatekey_length: The size of the private key in bytes.

Return: Returns qscabi_status_success on success, otherwise an error status.

qscabi_eddh_key_exchange

qscabi_status qscabi_eddh_key_exchange(uint8_t* secret, size_t secret_length, const uint8_t* privatekey, size_t privatekey_length, const uint8_t* publickey, size_t publickey_length);

Description: Computes an EDDH shared secret.

Parameters: secret: The caller-owned output buffer that receives the shared secret.; secret_length: The size of the shared-secret buffer in bytes.; privatekey: The caller-owned input local private key.; privatekey_length: The size of the private key in bytes.; publickey: The caller-owned input peer public key.; publickey_length: The size of the public key in bytes.

Return: Returns qscabi_status_success on success, otherwise an error status.

qscabi_ecdsa_public_key_size

size_t qscabi_ecdsa_public_key_size(void);

Description: Gets the ECDSA public-key size.

Return: The required ECDSA public-key size in bytes.

qscabi_ecdsa_private_key_size

size_t qscabi_ecdsa_private_key_size(void);

Description: Gets the ECDSA private-key size.

Return: The required ECDSA private-key size in bytes.

qscabi_ecdsa_signature_size

size_t qscabi_ecdsa_signature_size(void);

Description: Gets the ECDSA signature size.

Return: The ECDSA signature size in bytes.

qscabi_ecdsa_seed_size

size_t qscabi_ecdsa_seed_size(void);

Description: Gets the ECDSA deterministic seed size.

Return: The seed size in bytes required by qscabi_ecdsa_generate_seeded_keypair.

qscabi_ecdsa_signed_message_size

size_t qscabi_ecdsa_signed_message_size(size_t message_length);

Description: Calculates the ECDSA signed-message size for a message length.

Parameters: message_length: The message size in bytes.

Return: The signed-message size in bytes.

qscabi_ecdsa_generate_keypair

qscabi_status qscabi_ecdsa_generate_keypair(uint8_t* publickey, size_t publickey_length, uint8_t* privatekey, size_t privatekey_length);

Description: Generates an ECDSA key pair.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public-key buffer in bytes.; privatekey: The caller-owned output buffer that receives the private key.; privatekey_length: The size of the private-key buffer in bytes.

Return: Returns qscabi_status_success on success, otherwise an error status.

qscabi_ecdsa_generate_seeded_keypair

qscabi_status qscabi_ecdsa_generate_seeded_keypair(uint8_t* publickey, size_t publickey_length, uint8_t* privatekey, size_t privatekey_length, const uint8_t* seed, size_t seed_length);

Description: Generates a deterministic ECDSA key pair from a seed.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public-key buffer in bytes.; privatekey: The caller-owned output buffer that receives the private key.; privatekey_length: The size of the private-key buffer in bytes.; seed: The caller-owned input seed.; seed_length: The size of the seed in bytes.

Return: Returns qscabi_status_success on success, otherwise an error status.

qscabi_ecdsa_public_from_private

qscabi_status qscabi_ecdsa_public_from_private(uint8_t* publickey, size_t publickey_length, const uint8_t* privatekey, size_t privatekey_length);

Description: Derives an ECDSA public key from a private key.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public-key buffer in bytes.; privatekey: The caller-owned input private key.; privatekey_length: The size of the private key in bytes.

Return: Returns qscabi_status_success on success, otherwise an error status.

qscabi_ecdsa_sign

qscabi_status qscabi_ecdsa_sign(uint8_t* signed_message, size_t signed_message_length, size_t* written, const uint8_t* message, size_t message_length, const uint8_t* privatekey, size_t privatekey_length);

Description: Signs a message with ECDSA and writes a signed-message buffer.

Details: The signed_message buffer must be sized using qscabi_ecdsa_signed_message_size(message_length). The written parameter receives the number of bytes written to signed_message.

Parameters: signed_message: The caller-owned output buffer that receives the signed message.; signed_message_length: The size of the signed-message buffer in bytes.; written: Receives the number of bytes written to the signed-message buffer.; message: The caller-owned input message.; message_length: The size of the message in bytes.; privatekey: The caller-owned input private signing key.; privatekey_length: The size of the private key in bytes.

Return: Returns qscabi_status_success on success, otherwise an error status.

qscabi_ecdsa_verify

qscabi_status qscabi_ecdsa_verify(uint8_t* message, size_t message_length, size_t* written, const uint8_t* signed_message, size_t signed_message_length, const uint8_t* publickey, size_t publickey_length);

Description: Verifies an ECDSA signed message and recovers the original message.

Details: The message buffer must be large enough to receive the recovered message. The written parameter receives the number of bytes written to message.

Parameters: message: The caller-owned output buffer that receives the recovered message.; message_length: The size of the message output buffer in bytes.; written: Receives the number of bytes written to the message output buffer.; signed_message: The caller-owned input signed-message

buffer.; signed_message_length: The size of the signed-message buffer in bytes.; publickey: The caller-owned input public verification key.; publickey_length: The size of the public key in bytes.

Return: Returns qscabi_status_success on successful verification, qscabi_status_verification_failure when verification fails, otherwise an error status.

qscabi_eddsa_public_key_size

size_t qscabi_eddsa_public_key_size(void);

Description: Gets the EDDSA public-key size.

Return: The required EDDSA public-key size in bytes.

qscabi_eddsa_private_key_size

size_t qscabi_eddsa_private_key_size(void);

Description: Gets the EDDSA private-key size.

Return: The required EDDSA private-key size in bytes.

qscabi_eddsa_signature_size

size_t qscabi_eddsa_signature_size(void);

Description: Gets the EDDSA signature size.

Return: The EDDSA signature size in bytes.

qscabi_eddsa_seed_size

size_t qscabi_eddsa_seed_size(void);

Description: Gets the EDDSA deterministic seed size.

Return: The seed size in bytes required by qscabi_eddsa_generate_seeded_keypair.

qscabi_eddsa_signed_message_size

size_t qscabi_eddsa_signed_message_size(size_t message_length);

Description: Calculates the EDDSA signed-message size for a message length.

Parameters: message_length: The message size in bytes.

Return: The signed-message size in bytes.

qscabi_eddsa_generate_keypair

qscabi_status qscabi_eddsa_generate_keypair(uint8_t* publickey, size_t publickey_length, uint8_t* privatekey, size_t privatekey_length);

Description: Generates an EDDSA key pair.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public-key buffer in bytes.; privatekey: The caller-owned output buffer that receives the private key.; privatekey_length: The size of the private-key buffer in bytes.

Return: Returns qscabi_status_success on success, otherwise an error status.

qscabi_eddsa_generate_seeded_keypair

qscabi_status qscabi_eddsa_generate_seeded_keypair(uint8_t* publickey, size_t publickey_length, uint8_t* privatekey, size_t privatekey_length, const uint8_t* seed, size_t seed_length);

Description: Generates a deterministic EDDSA key pair from a seed.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public-key buffer in bytes.; privatekey: The caller-owned output buffer that receives the private key.; privatekey_length: The size of the private-key buffer in bytes.; seed: The caller-owned input seed.; seed_length: The size of the seed in bytes.

Return: Returns qscabi_status_success on success, otherwise an error status.

qscabi_eddsa_public_from_private

qscabi_status qscabi_eddsa_public_from_private(uint8_t* publickey, size_t publickey_length, const uint8_t* privatekey, size_t privatekey_length);

Description: Derives an EDDSA public key from an existing private key.

Details: The QSC EDDSA private-key representation stores the deterministic seed and public key material. This ABI helper derives the public key from the seed portion of the private key and writes the resulting public verification key into caller-owned output storage.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public-key buffer in bytes.; privatekey: The caller-owned input private signing key.; privatekey_length: The size of the private-key buffer in bytes.

Return: Returns qscabi_status_success on success, otherwise an error status.

qscabi_eddsa_sign

qscabi_status qscabi_eddsa_sign(uint8_t* signed_message, size_t signed_message_length, size_t* written, const uint8_t* message, size_t message_length, const uint8_t* privatekey, size_t privatekey_length);

Description: Signs a message with EDDSA and writes a signed-message buffer.

Details: The signed_message buffer must be sized using qscabi_eddsa_signed_message_size(message_length). The written parameter receives the number of bytes written to signed_message.

Parameters: signed_message: The caller-owned output buffer that receives the signed message.; signed_message_length: The size of the signed-message buffer in bytes.; written: Receives the number of bytes written to the signed-message buffer.; message: The caller-owned input message.; message_length: The size of the message in bytes.; privatekey: The caller-owned input private signing key.; privatekey_length: The size of the private key in bytes.

Return: Returns qscabi_status_success on success, otherwise an error status.

qscabi_eddsa_verify

qscabi_status qscabi_eddsa_verify(uint8_t* message, size_t message_length, size_t* written, const uint8_t* signed_message, size_t signed_message_length, const uint8_t* publickey, size_t publickey_length);

Description: Verifies an EDDSA signed message and recovers the original message.

Details: The message buffer must be large enough to receive the recovered message. The written parameter receives the number of bytes written to message.

Parameters: message: The caller-owned output buffer that receives the recovered message.; message_length: The size of the message output buffer in bytes.; written: Receives the number of bytes written to the message output buffer.; signed_message: The caller-owned input signed-message buffer.; signed_message_length: The size of the signed-message buffer in bytes.; publickey: The caller-owned input public verification key.; publickey_length: The size of the public key in bytes.

Return: Returns qscabi_status_success on successful verification, qscabi_status_verification_failure when verification fails, otherwise an error status.

23.2 qscabi_encoding.h — 12 functions

Hex/Base64/PEM conversion with caller-sized buffers.

qscabi_hex_encoded_size

size_t qscabi_hex_encoded_size(size_t input_length);

Description: Returns the hexadecimal character count required for a binary buffer.

Parameters: input_length: [size_t] Number of binary input bytes to encode.

Return: Returns the number of hexadecimal characters required to encode the input, excluding a terminating null character. Returns zero if the requested calculation would overflow size_t.

Notes: Callers must allocate at least the returned value plus one additional byte before calling qscabi_hex_encode(), because the encoder writes a null terminator after the hexadecimal text.

qscabi_hex_decoded_size

qscabi_status qscabi_hex_decoded_size(size_t* output_length, const char* input, size_t input_length);

Description: Computes the binary output size for a hexadecimal string.

Parameters: output_length: [size_t*] Pointer that receives the decoded byte count.; input: [const char*] Pointer to the hexadecimal input string.; input_length: [size_t] Length of input in characters.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code indicating invalid arguments or an invalid hexadecimal length.

Notes: Hexadecimal input must contain an even number of characters. When input_length is zero, input may be NULL and the decoded size is zero.

qscabi_hex_encode

qscabi_status qscabi_hex_encode(char* output, size_t output_length, size_t* written, const uint8_t* input, size_t input_length);

Description: Encodes binary data as null-terminated hexadecimal text.

Parameters: output: [char*] Caller-owned output buffer that receives the encoded text.; output_length: [size_t] Size of the output buffer in bytes.; written: [size_t*] Pointer that receives the number of encoded characters, excluding the null terminator.; input: [const uint8_t*] Caller-owned binary input buffer.; input_length: [size_t] Length of input in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

Notes: The output buffer must be larger than qscabi_hex_encoded_size() so the null terminator can be written. For zero-length input, the function writes an empty string and reports zero characters written.

qscabi_hex_decode

qscabi_status qscabi_hex_decode(uint8_t* output, size_t output_length, size_t* written, const char* input, size_t input_length);

Description: Decodes hexadecimal text into binary data.

Parameters: output: [uint8_t*] Caller-owned output buffer that receives decoded bytes.; output_length: [size_t] Size of the output buffer in bytes.; written: [size_t*] Pointer that receives the number of decoded bytes written.; input: [const char*] Pointer to the hexadecimal input string.; input_length: [size_t] Length of input in characters.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

Notes: The caller should first call qscabi_hex_decoded_size() and provide an output buffer at least as large as the returned value.

qscabi_base64_encoded_size

size_t qscabi_base64_encoded_size(size_t input_length);

Description: Returns the Base64 character count required for a binary buffer.

Parameters: input_length: [size_t] Number of binary input bytes to encode.

Return: Returns the number of RFC 4648 Base64 characters required to encode the input, excluding a terminating null character. Returns zero for zero-length input.

qscabi_base64_decoded_size

qscabi_status qscabi_base64_decoded_size(size_t* output_length, const char* input, size_t input_length);

Description: Computes the binary output size for a Base64 string.

Parameters: output_length: [size_t*] Pointer that receives the decoded byte count.; input: [const char*] Pointer to the Base64 input string.; input_length: [size_t] Length of input in characters.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

Notes: The Base64 input length must be a multiple of four characters unless it is zero. Whitespace folding is not handled by this raw Base64 helper; callers should use PEM decoding when working with armored multi-line material.

qscabi_base64_encode

qscabi_status qscabi_base64_encode(char* output, size_t output_length, size_t* written, const uint8_t* input, size_t input_length);

Description: Encodes binary data as null-terminated RFC 4648 Base64 text.

Parameters: output: [char*] Caller-owned output buffer that receives encoded text.; output_length: [size_t] Size of the output buffer in bytes.; written: [size_t*] Pointer that receives the number of Base64 characters written, excluding the null terminator.; input: [const uint8_t*] Caller-owned binary input buffer.; input_length: [size_t] Length of input in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

Notes: The output buffer must be larger than qscabi_base64_encoded_size() so the null terminator can be written.

qscabi_base64_decode

qscabi_status qscabi_base64_decode(uint8_t* output, size_t output_length, size_t* written, const char* input, size_t input_length);

Description: Decodes RFC 4648 Base64 text into binary data.

Parameters: output: [uint8_t*] Caller-owned output buffer that receives decoded bytes.; output_length: [size_t] Size of the output buffer in bytes.; written: [size_t*] Pointer that receives the number of decoded bytes written.; input: [const char*] Pointer to the Base64 input string.; input_length: [size_t] Length of input in characters.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

qscabi_pem_encoded_size

qscabi_status qscabi_pem_encoded_size(size_t* output_length, const char* label, size_t data_length);

Description: Computes the character count required for a PEM document.

Parameters: output_length: [size_t*] Pointer that receives the PEM character count, excluding the null terminator.; label: [const char*] Null-terminated PEM object label.; data_length: [size_t] Binary payload length in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

Notes: The size includes the PEM armor header, line-folded Base64 body, footer, and required newlines, but excludes the final null terminator written by qscabi_pem_encode().

qscabi_pem_decoded_size

qscabi_status qscabi_pem_decoded_size(size_t* output_length, const char* input, size_t input_length);

Description: Computes the binary payload size for a PEM document.

Parameters: output_length: [size_t*] Pointer that receives the decoded payload size.; input: [const char*] Pointer to the PEM document text.; input_length: [size_t] Length of input in characters.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

Notes: This function extracts the Base64 body from the PEM armor and computes the decoded payload size. Header/footer lines are skipped during collection.

qscabi_pem_encode

qscabi_status qscabi_pem_encode(char* output, size_t output_length, size_t* written, const char* label, const uint8_t* data, size_t data_length);

Description: Encodes binary data into a null-terminated PEM document.

Parameters: output: [char*] Caller-owned output buffer that receives PEM text.; output_length: [size_t] Size of the output buffer in bytes.; written: [size_t*] Pointer that receives the number of PEM characters written, excluding the null terminator.; label: [const char*] Null-terminated PEM object label.; data: [const uint8_t*] Caller-owned binary payload buffer.; data_length: [size_t] Length of data in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

Notes: The PEM label is validated by QSCABI and must not contain whitespace or hyphen characters under the current ABI validation policy.

qscabi_pem_decode

qscabi_status qscabi_pem_decode(uint8_t* output, size_t output_length, size_t* written, const char* input, size_t input_length);

Description: Decodes a PEM document into binary payload data.

Parameters: output: [uint8_t*] Caller-owned output buffer that receives the decoded payload.; output_length: [size_t] Size of the output buffer in bytes.; written: [size_t*] Pointer that receives the number of decoded bytes written.; input: [const char*] Pointer to the PEM document text.; input_length: [size_t] Length of input in characters.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

Notes: The caller should first call qscabi_pem_decoded_size() and provide an output buffer at least as large as the returned value.

23.3 qscabi_entropy.h — 10 functions

ACP, CSP, RDP and CJP random source interfaces.

qscabi_entropy_acp_seed_max

size_t qscabi_entropy_acp_seed_max(void);

Description: Gets the maximum ACP generation request size.

Details: This function returns the largest output length accepted by qscabi_entropy_acp_generate.

Return: The ACP provider maximum request size in bytes.

qscabi_entropy_csp_seed_max

size_t qscabi_entropy_csp_seed_max(void);

Description: Gets the maximum CSP generation request size.

Details: This function returns the largest output length accepted by qscabi_entropy_csp_generate.

Return: The CSP provider maximum request size in bytes.

qscabi_entropy_rdp_seed_max

size_t qscabi_entropy_rdp_seed_max(void);

Description: Gets the maximum RDP generation request size.

Details: This function returns the largest output length accepted by qscabi_entropy_rdp_generate.

Return: The RDP provider maximum request size in bytes.

qscabi_entropy_cjp_seed_max

size_t qscabi_entropy_cjp_seed_max(void);

Description: Gets the maximum CJP generation request size.

Details: This function returns the largest output length accepted by qscabi_entropy_cjp_generate.

Return: The CJP provider maximum request size in bytes.

qscabi_entropy_rdp_available

int qscabi_entropy_rdp_available(void);

Description: Tests whether the RDP provider is available.

Details: The RDP provider depends on hardware random-data support. This function reports whether the provider is available on the current platform and build.

Return: Returns a non-zero value when RDP is available; otherwise returns zero.

qscabi_entropy_cjp_available

int qscabi_entropy_cjp_available(void);

Description: Tests whether the CJP provider is available.

Details: The CJP provider depends on CPU-jitter collection support. This function reports whether the provider is available on the current platform and build.

Return: Returns a non-zero value when CJP is available; otherwise returns zero.

qscabi_entropy_acp_generate

qscabi_status qscabi_entropy_acp_generate(uint8_t* output, size_t output_length);

Description: Generates bytes from the aggregate entropy provider.

Details: This function fills a caller-owned output buffer using the QSC aggregate entropy-provider path. The output pointer must be non-null when output_length is greater than zero. QSCABI does not retain the output pointer after return.

Parameters: output: The caller-owned output buffer that receives generated bytes.; output_length: The number of bytes to generate.

Return: Returns qscabi_status_success on success; otherwise returns qscabi_status_invalid_argument, qscabi_status_invalid_size, or qscabi_status_failure.

Notes: ACP output is appropriate for seed material when the aggregate provider succeeds. Do not expose generated bytes in logs or diagnostic output.

qscabi_entropy_csp_generate

qscabi_status qscabi_entropy_csp_generate(uint8_t* output, size_t output_length);

Description: Generates bytes from the operating-system cryptographic provider.

Details: This function fills a caller-owned output buffer using the platform cryptographic service provider. The output pointer must be non-null when output_length is greater than zero. If the platform provider cannot satisfy the request, the function returns qscabi_status_platform_unavailable or qscabi_status_failure according to the underlying failure mode.

Parameters: output: The caller-owned output buffer that receives generated bytes.; output_length: The number of bytes to generate.

Return: Returns qscabi_status_success on success; otherwise returns qscabi_status_invalid_argument, qscabi_status_invalid_size, qscabi_status_platform_unavailable, or qscabi_status_failure.

Notes: CSP output is the preferred operating-system entropy source for wrapper-level random seed generation. Do not treat provider availability as proof of application-level key-management correctness.

qscabi_entropy_rdp_generate

qscabi_status qscabi_entropy_rdp_generate(uint8_t* output, size_t output_length);

Description: Generates bytes from the hardware random-data provider.

Details: This function fills a caller-owned output buffer using the RDP hardware provider. Call qscabi_entropy_rdp_available before relying on this provider. If the provider is unavailable, the function returns qscabi_status_platform_unavailable.

Parameters: output: The caller-owned output buffer that receives generated bytes.; output_length: The number of bytes to generate.

Return: Returns qscabi_status_success on success; otherwise returns qscabi_status_invalid_argument, qscabi_status_invalid_size, qscabi_status_platform_unavailable, or qscabi_status_failure.

Notes: RDP is hardware dependent; callers should check qscabi_entropy_rdp_available before requiring this provider. Hardware-provider output should still be domain-separated before use in protocol keys.

qscabi_entropy_cjp_generate

qscabi_status qscabi_entropy_cjp_generate(uint8_t* output, size_t output_length);

Description: Generates bytes from the CPU-jitter provider.

Details: This function fills a caller-owned output buffer using the CJP timing-jitter provider. Call qscabi_entropy_cjp_available before relying on this provider. If the provider is unavailable, the function returns qscabi_status_platform_unavailable.

Parameters: output: The caller-owned output buffer that receives generated bytes.; output_length: The number of bytes to generate.

Return: Returns qscabi_status_success on success; otherwise returns qscabi_status_invalid_argument, qscabi_status_invalid_size, qscabi_status_platform_unavailable, or qscabi_status_failure.

Notes: CJP is platform and timing-environment dependent; callers should check qscabi_entropy_cjp_available before requiring this provider. CJP should be used as an entropy source, not as an application-visible random-number generator interface.

23.4 qscabi_explicit_curve.h — 5 functions

Runtime-selected curve key generation and agreement.

qscabi_curve_get_sizes

qscabi_status qscabi_curve_get_sizes(qscabi_curve_algorithm algorithm, qscabi_curve_sizes* sizes);

Description: Query the native key and seed lengths of a named curve.

Parameters: algorithm: Named curve selection.; sizes [out]: Exact sizes.

Return: Success or unsupported, clears supplied sizes on failure.

qscabi_curve_generate

qscabi_status qscabi_curve_generate(qscabi_curve_algorithm algorithm, uint8_t* public_key, size_t public_length, uint8_t* private_key, size_t private_length);

Description: Generate keys with QSC CSP entropy and native seeded generation.

Parameters: algorithm: Named curve.; public_key [out]: Caller buffer.; public_length: Exact public bytes.; private_key [out]: Secret result.; private_length: Exact private bytes.

Return: Status; secrets erased on failure.

qscabi_curve_generate_seeded

qscabi_status qscabi_curve_generate_seeded(qscabi_curve_algorithm algorithm, uint8_t* public_key, size_t public_length, uint8_t* private_key, size_t private_length, const uint8_t* seed, size_t seed_length);

Description: Deterministically generate a curve keypair using caller-provided seed.

Parameters: algorithm: Named curve.; public_key [out]: Public bytes.; public_length: Public buffer size.; private_key [out]: Secret bytes.; private_length: Secret buffer size.; seed: Exact native seed.; seed_length: Seed size.

Return: Status; output erasure on failure.

qscabi_curve_public_from_private

qscabi_status qscabi_curve_public_from_private(qscabi_curve_algorithm algorithm, uint8_t* public_key, size_t public_length, const uint8_t* private_key, size_t private_length);

Description: Derive an uncompressed native public point from the private scalar.

Parameters: algorithm: Curve.; public_key [out]: Output.; public_length: Public bytes.; private_key: Secret native scalar.; private_length: Secret size.

Return: Success or validation/native failure.

qscabi_curve_agree

qscabi_status qscabi_curve_agree(qscabi_curve_algorithm algorithm, uint8_t* secret, size_t secret_length, const uint8_t* peer_public_key, size_t peer_length, const uint8_t* private_key, size_t private_length);

Description: Perform checked ECDH/EDDH agreement and reject invalid peer points.

Parameters: algorithm: Curve.; secret [out]: Caller output (cleared on failure).; secret_length: Secret bytes.; peer_public_key: Peer native point.; peer_length: Peer public-key size.; private_key: Local private scalar.; private_length: Local secret size.

Return: Status.

23.5 qscabi_explicit_kem.h — 4 functions

Runtime-selected KEM key generation and encapsulation.

qscabi_kem_get_sizes

qscabi_status qscabi_kem_get_sizes(qscabi_kem_algorithm algorithm, qscabi_kem_sizes* sizes);

Description: Query exact key, ciphertext and secret sizes for a named KEM.

Parameters: algorithm: Stable parameter-set identifier.; sizes [out]: Caller-owned output; cleared on an invalid identifier.

Return: Success or unsupported; safe for independent concurrent calls.

qscabi_kem_generate

qscabi_status qscabi_kem_generate(qscabi_kem_algorithm algorithm, uint8_t* public_key, size_t public_length, uint8_t* private_key, size_t private_length);

Description: Generate an explicit native QSC KEM keypair with system CSP entropy.

Parameters: algorithm: Explicit parameter set.; public_key [out]: Exact-sized caller buffer.; public_length: Public-key bytes.; private_key [out]: Exact-sized caller buffer; cleared on RNG failure.; private_length: Secret-key bytes.

Return: Exact argument/size/failure result; caller owns and must erase key.

qscabi_kem_encapsulate

qscabi_status qscabi_kem_encapsulate(qscabi_kem_algorithm algorithm, uint8_t* ciphertext, size_t ciphertext_length, uint8_t* secret, size_t secret_length, const uint8_t* public_key, size_t public_length);

Description: Encapsulate a shared secret with the selected QSC KEM.

Parameters: algorithm: Explicit parameter set.; ciphertext [out]: Exact-sized caller output.; ciphertext_length: Output buffer size.; secret [out]: Exact-sized caller secret output; cleared on failure.; secret_length: Secret byte count.; public_key: Exact native public key input.; public_length: Input key size.

Return: Success or validation/entropy/native failure; outputs never overlap.

qscabi_kem_decapsulate

qscabi_status qscabi_kem_decapsulate(qscabi_kem_algorithm algorithm, uint8_t* secret, size_t secret_length, const uint8_t* ciphertext, size_t ciphertext_length, const uint8_t* private_key, size_t private_length);

Description: Derive a KEM secret with exact ciphertext and secret-key lengths.

Details: Invalid ciphertext may return a substituted secret under native implicit rejection, not an authentication failure. Never interpret success as proof of recipient or sender identity.

Parameters: algorithm: Explicit KEM configuration.; secret [out]: Caller-owned shared-secret output, cleared on failure.; secret_length: Shared-secret buffer size.; ciphertext: Ciphertext from encapsulation.; ciphertext_length: Exact input ciphertext bytes.; private_key: Secret recipient key.; private_length: Exact recipient key bytes.

Return: Native success translated to QSCABI status; independent calls are thread-safe.

23.6 qscabi_explicit_signature.h — 4 functions

Runtime-selected attached signature algorithms.

qscabi_signature_get_sizes

qscabi_status qscabi_signature_get_sizes(qscabi_attached_signature_algorithm algorithm, qscabi_signature_sizes* sizes);

Description: Retrieve native exact key lengths and maximum signature overhead.

Parameters: algorithm: Signature scheme.; sizes [out]: Caller metadata result.

Return: Success or unsupported; sizes zeroed on unsupported scheme.

qscabi_signature_generate

qscabi_status qscabi_signature_generate(qscabi_attached_signature_algorithm algorithm, uint8_t* public_key, size_t public_length, uint8_t* private_key, size_t private_length);

Description: Generate a signing keypair with the native CSP source.

Parameters: algorithm: Signature scheme.; public_key [out]: Caller public bytes.; public_length: Public size.; private_key [out]: Secret output.; private_length: Secret size.

Return: Status, clearing outputs on failure.

qscabi_signature_sign

qscabi_status qscabi_signature_sign(qscabi_attached_signature_algorithm algorithm, uint8_t* signed_message, size_t capacity, size_t* written, const uint8_t* message, size_t message_length, const uint8_t* private_key, size_t private_length);

Description: Sign message into native attached signed-message representation.

Parameters: algorithm: Explicit scheme.; signed_message [out]: Destination.; capacity: At least message_length + signature_max bytes; checked for overflow.; written [out]: Actual signed bytes (reset to zero on failure).; message: Nonempty borrowed input.; message_length: Input bytes.; private_key: Native signing key.; private_length: Exact key size.

Return: Status; signed output erased on native failure. No pointer aliasing.

qscabi_signature_verify

qscabi_status qscabi_signature_verify(qscabi_attached_signature_algorithm algorithm, uint8_t* message, size_t capacity, size_t* written, const uint8_t* signed_message, size_t signed_length, const uint8_t* public_key, size_t public_length);

Description: Verify native attached signature and recover signed message.

Parameters: algorithm: Exact signature scheme.; message [out]: Caller output.; capacity: At least signed_length bytes (conservative native verifier bound).; written [out]: Actual recovered bytes, zeroed on verification failure.; signed_message: Nonempty signed native input.; signed_length: Input bytes.; public_key: Native verification key.; public_length: Exact key bytes.

Return: Success only for an accepted native signature, otherwise failure.

23.7 qscabi_explicit_slhdsa.h — 4 functions

Explicit FIPS 205 variants, prehash and context-controlled signatures.

qscabi_slhdsa_get_sizes

qscabi_status qscabi_slhdsa_get_sizes(qscabi_slhdsa_algorithm algorithm, qscabi_slhdsa_sizes* sizes);

Description: Retrieve exact SLH-DSA parameter sizes.

Parameters: algorithm: Native variant.; sizes [out]: Caller metadata output.

Return: Success or unsupported; output zeroed on invalid variant.

qscabi_slhdsa_explicit_generate

qscabi_status qscabi_slhdsa_explicit_generate(qscabi_slhdsa_algorithm algorithm, uint8_t* public_key, size_t public_length, uint8_t* private_key, size_t private_length);

Description: Generate an explicit SLH-DSA keypair with native CSP.

Parameters: algorithm: Variant.; public_key [out]: Exact public output.; public_length: Public size.; private_key [out]: Secret output.; private_length: Secret size.

Return: Native status, clearing outputs on failure.

qscabi_slhdsa_explicit_sign

qscabi_status qscabi_slhdsa_explicit_sign(qscabi_slhdsa_algorithm algorithm, qscabi_slhdsa_prehash prehash, bool randomized, uint8_t* signature, size_t signature_length, const uint8_t* message, size_t message_length, const uint8_t* context, size_t context_length, const uint8_t* private_key, size_t private_length);

Description: Sign with pure or prehash FIPS 205 context binding.

Parameters: algorithm: SLH-DSA variant.; prehash: Pure or named prehash.; randomized: Use native CSP signing randomness if true, deterministic otherwise.; signature [out]: Exact-sized detached output, cleared on failure.; signature_length: Exact signature capacity.; message: Nonempty signed bytes.; message_length: Message bytes.; context: Optional domain context.; context_length: At most 255 bytes.; private_key: Exact secret key.; private_length: Native key bytes.

Return: Success or ABI error.

qscabi_slhdsa_explicit_verify

qscabi_status qscabi_slhdsa_explicit_verify(qscabi_slhdsa_algorithm algorithm, qscabi_slhdsa_prehash prehash, const uint8_t* signature, size_t signature_length, const uint8_t* message, size_t message_length, const uint8_t* context, size_t context_length, const uint8_t* public_key, size_t public_length);

Description: Verify a native FIPS 205 detached signature and context.

Parameters: algorithm: Variant.; prehash: Pure or prehash mode.; signature: Exact detached signature.; signature_length: Signature bytes.; message: Nonempty message.; message_length: Message bytes.; context: Optional context.; context_length: At most 255.; public_key: Exact public key.; public_length: Key bytes.

Return: Success only for correct signature, else verification failure or ABI error.

23.8 qscabi_hardware.h — 14 functions

Hardware provider callbacks and opaque hardware key handles.

qscabi_hw_is_enabled

int qscabi_hw_is_enabled(void);

Description: Report whether this QSCABI binary was linked to a hardware-enabled QSC.

Return: 1 when enabled, 0 otherwise. No device presence is implied.

qscabi_hw_provider_open

qscabi_status qscabi_hw_provider_open(qscabi_hw_provider** provider, const qscabi_hw_backend_ops* operations, void* context);

Description: Open an ABI provider bound to caller-managed callback state.

Parameters: provider: Receives live owned handle or null.; operations: Borrowed callback table copied into this handle.; context: Borrowed backend context, valid through close.

Return: Success, invalid input, unsupported, or allocation failure.

qscabi_hw_provider_capabilities

qscabi_status qscabi_hw_provider_capabilities(qscabi_hw_provider* provider, qscabi_hw_capabilities* capabilities);

Description: Query current backend capabilities without attesting to hardware custody.

Parameters: provider: Live provider.; capabilities: Caller-owned descriptor initialized with size/version.

Return: Mapped ABI status.

qscabi_hw_key_open

qscabi_status qscabi_hw_key_open(qscabi_hw_key** key, qscabi_hw_provider* provider, const char* locator, const qscabi_hw_policy* policy);

Description: Open a persistent key subject to immutable policy.

Parameters: key: Receives transient reference, or null.; provider: Live provider.; locator: NUL-terminated key locator, borrowed for this call.; policy: Caller versioned policy; no private bytes returned.

Return: Mapped status, including policy rejection.

qscabi_hw_key_generate

qscabi_status qscabi_hw_key_generate(qscabi_hw_key** key, qscabi_hw_provider* provider, const qscabi_hw_policy* policy);

Description: Generate a new key according to immutable policy.

Parameters: key: Receives transient handle or null.; provider: Live provider.; policy: Caller versioned policy.

Return: Mapped status.

qscabi_hw_key_get_info

qscabi_status qscabi_hw_key_get_info(const qscabi_hw_key* key, qscabi_hw_key_info* info);

Description: Get public key algorithm/custody information.

Parameters: key: Live key reference.; info: Caller versioned metadata buffer.

Return: Mapped status.

qscabi_hw_key_public

qscabi_status qscabi_hw_key_public(qscabi_hw_key* key, uint8_t* output, size_t* output_length);

Description: Retrieve public key bytes or query required size.

Parameters: key: Live key reference.; output: Caller public buffer or null for size query.; output_length: In/out byte capacity or required size.

Return: Mapped status.

qscabi_hw_key_sign

qscabi_status qscabi_hw_key_sign(qscabi_hw_key* key, const uint8_t* message, size_t message_length, uint8_t* signature, size_t* signature_length);

Description: Sign bytes without exporting private-key material.

Parameters: key: Live key reference.; message: Borrowed message bytes, nullable only if zero length.; message_length: Number of bytes.; signature: Caller output or null for size query.; signature_length: In/out capacity and actual/required bytes.

Return: Mapped status. Output cleared on non-size errors by native QSC.

qscabi_hw_key_agree

qscabi_status qscabi_hw_key_agree(qscabi_hw_key* key, const uint8_t* peer, size_t peer_length, uint8_t* secret, size_t* secret_length);

Description: Export a derived raw shared secret only when explicitly permitted.

Parameters: key: Live key reference.; peer: Borrowed peer public key bytes.; peer_length: Number of input bytes.; secret: Caller output or null for size query.; secret_length: In/out capacity/actual secret length.

Return: Mapped status, including policy denial.

qscabi_hw_key_attest

qscabi_status qscabi_hw_key_attest(qscabi_hw_key* key, const uint8_t* challenge, size_t challenge_length, uint8_t* evidence, size_t* evidence_length);

Description: Retrieve backend-specific attestation evidence; no verification implied.

Parameters: key: Live key reference.; challenge: Borrowed, nonempty challenge.; challenge_length: Number of challenge bytes.; evidence: Caller output or null for size query.; evidence_length: In/out output capacity.

Return: Mapped status. Evidence must be independently validated.

qscabi_hw_provider_random

qscabi_status qscabi_hw_provider_random(qscabi_hw_provider* provider, uint8_t* output, size_t length);

Description: Retrieve bytes from a selected hardware/backend RNG.

Parameters: provider: Live provider.; output: Caller buffer; nullable only for zero length.; length: Output byte count.

Return: Mapped status; output zeroized on failure by QSC.

qscabi_hw_provider_invalidate

qscabi_status qscabi_hw_provider_invalidate(qscabi_hw_provider* provider);

Description: Mark all provider keys stale after disconnect/reconnect.

Parameters: provider: Live provider whose key references become invalid.

Return: Mapped status.

qscabi_hw_key_release

qscabi_status qscabi_hw_key_release(qscabi_hw_key** key);

Description: Release transient key ownership and clear the caller's pointer.

Parameters: key: Pointer to owned key, set null on success.

Return: Mapped status.

qscabi_hw_provider_close

qscabi_status qscabi_hw_provider_close(qscabi_hw_provider** provider);

Description: Close a provider only after every transient key is released.

Parameters: provider: Pointer to owned handle, set null on successful close.

Return: Busy while references remain; backend context never freed by ABI.

23.9 qscabi_hash.h — 12 functions

SHA-2 and SHA-3 one-shot hashes.

qscabi_sha2_256_digest_size

size_t qscabi_sha2_256_digest_size(void);

Description: Returns the SHA2-256 digest size in bytes.

Return: Returns QSCABI_SHA2_256_DIGEST_SIZE.

qscabi_sha2_384_digest_size

size_t qscabi_sha2_384_digest_size(void);

Description: Returns the SHA2-384 digest size in bytes.

Return: Returns QSCABI_SHA2_384_DIGEST_SIZE.

qscabi_sha2_512_digest_size

size_t qscabi_sha2_512_digest_size(void);

Description: Returns the SHA2-512 digest size in bytes.

Return: Returns QSCABI_SHA2_512_DIGEST_SIZE.

qscabi_sha3_128_digest_size

size_t qscabi_sha3_128_digest_size(void);

Description: Returns the QSC SHA3-128 digest size in bytes.

Return: Returns QSCABI_SHA3_128_DIGEST_SIZE.

qscabi_sha3_256_digest_size

size_t qscabi_sha3_256_digest_size(void);

Description: Returns the SHA3-256 digest size in bytes.

Return: Returns QSCABI_SHA3_256_DIGEST_SIZE.

qscabi_sha3_512_digest_size

size_t qscabi_sha3_512_digest_size(void);

Description: Returns the SHA3-512 digest size in bytes.

Return: Returns QSCABI_SHA3_512_DIGEST_SIZE.

qscabi_sha2_256_compute

qscabi_status qscabi_sha2_256_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length);

Description: Computes a SHA2-256 message digest.

Parameters: output: [uint8_t*] Caller-owned output buffer that receives the digest.; output_length: [size_t] Size of output in bytes; must be at least qscabi_sha2_256_digest_size().; message: [const uint8_t*] Caller-owned message buffer.; message_length: [size_t] Length of message in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

qscabi_sha2_384_compute

qscabi_status qscabi_sha2_384_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length);

Description: Computes a SHA2-384 message digest.

Parameters: output: [uint8_t*] Caller-owned output buffer that receives the digest.; output_length: [size_t] Size of output in bytes; must be at least qscabi_sha2_384_digest_size().; message: [const uint8_t*] Caller-owned message buffer.; message_length: [size_t] Length of message in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

qscabi_sha2_512_compute

qscabi_status qscabi_sha2_512_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length);

Description: Computes a SHA2-512 message digest.

Parameters: output: [uint8_t*] Caller-owned output buffer that receives the digest.; output_length: [size_t] Size of output in bytes; must be at least qscabi_sha2_512_digest_size().; message: [const uint8_t*] Caller-owned message buffer.; message_length: [size_t] Length of message in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

qscabi_sha3_128_compute

qscabi_status qscabi_sha3_128_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length);

Description: Computes a QSC SHA3-128 message digest.

Parameters: output: [uint8_t*] Caller-owned output buffer that receives the digest.; output_length: [size_t] Size of output in bytes; must be at least qscabi_sha3_128_digest_size().; message: [const uint8_t*] Caller-owned message buffer.; message_length: [size_t] Length of message in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

Notes: This QSCABI helper exposes the QSC SHA3-128 digest variant with a 16-byte output length. Use SHA3-256 or SHA3-512 when a standard-strength digest size is required by an external protocol profile.

qscabi_sha3_256_compute

qscabi_status qscabi_sha3_256_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length);

Description: Computes a SHA3-256 message digest.

Parameters: output: [uint8_t*] Caller-owned output buffer that receives the digest.; output_length: [size_t] Size of output in bytes; must be at least qscabi_sha3_256_digest_size().; message: [const uint8_t*] Caller-owned message buffer.; message_length: [size_t] Length of message in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

qscabi_sha3_512_compute

qscabi_status qscabi_sha3_512_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length);

Description: Computes a SHA3-512 message digest.

Parameters: output: [uint8_t*] Caller-owned output buffer that receives the digest.; output_length: [size_t] Size of output in bytes; must be at least qscabi_sha3_512_digest_size().; message: [const uint8_t*] Caller-owned message buffer.; message_length: [size_t] Length of message in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

23.10 qscabi_kdf.h — 12 functions

HKDF and PBMAC1 key derivation.

qscabi_hkdf_sha2_256_prk_size

size_t qscabi_hkdf_sha2_256_prk_size(void);

Description: Returns the HKDF-SHA2-256 PRK size.

Return: The HKDF-SHA2-256 pseudorandom key size in bytes.

qscabi_hkdf_sha2_384_prk_size

size_t qscabi_hkdf_sha2_384_prk_size(void);

Description: Returns the HKDF-SHA2-384 PRK size.

Return: The HKDF-SHA2-384 pseudorandom key size in bytes.

qscabi_hkdf_sha2_512_prk_size

size_t qscabi_hkdf_sha2_512_prk_size(void);

Description: Returns the HKDF-SHA2-512 PRK size.

Return: The HKDF-SHA2-512 pseudorandom key size in bytes.

qscabi_pbmac1_max_key_size

size_t qscabi_pbmac1_max_key_size(void);

Description: Returns the maximum PBMAC1 derived-key size.

Return: The maximum PBMAC1 derived-key size in bytes.

qscabi_pbmac1_max_iterations

uint64_t qscabi_pbmac1_max_iterations(void);

Description: Returns the maximum accepted PBMAC1 iteration count.

Return: The maximum PBMAC1 iteration count accepted by the ABI.

qscabi_hkdf_sha2_256_extract

qscabi_status qscabi_hkdf_sha2_256_extract(uint8_t* output, size_t output_length, const uint8_t* key, size_t key_length, const uint8_t* salt, size_t salt_length);

Description: Performs HKDF-SHA2-256 extract.

Details: This function derives a 32-byte HKDF pseudorandom key from input keying material and an optional salt. The output buffer must be at least see QSCABI_HKDF_SHA2_256_PRK_SIZE bytes.

Parameters: output [out]: The caller-owned output buffer that receives the PRK.; output_length [in]: The size of the output buffer in bytes.; key [in]: The caller-owned input keying-material buffer.; key_length [in]: The size of the input keying-material buffer in bytes.; salt [in]: The optional caller-owned salt buffer, or NULL when salt_length is zero.; salt_length [in]: The size of the salt buffer in bytes.

Return: Returns see qscabi_status_success on success, otherwise a status code indicating the validation or derivation failure.

qscabi_hkdf_sha2_256_expand

qscabi_status qscabi_hkdf_sha2_256_expand(uint8_t* output, size_t output_length, const uint8_t* key, size_t key_length, const uint8_t* info, size_t info_length);

Description: Performs HKDF-SHA2-256 expand.

Details: This function expands an HKDF-SHA2-256 PRK into caller-requested output using optional context information. The key buffer should contain a PRK produced by see qscabi_hkdf_sha2_256_extract or an equivalent HKDF-SHA2-256 extract step.

Parameters: output [out]: The caller-owned output buffer that receives the derived bytes.; output_length [in]: The number of output bytes to derive.; key [in]: The caller-owned PRK buffer.; key_length [in]: The size of the PRK buffer in bytes.; info [in]: The optional caller-owned context-information buffer, or NULL when info_length is zero.; info_length [in]: The size of the context-information buffer in bytes.

Return: Returns see qscabi_status_success on success, otherwise a status code indicating the validation or derivation failure.

qscabi_hkdf_sha2_384_extract

qscabi_status qscabi_hkdf_sha2_384_extract(uint8_t* output, size_t output_length, const uint8_t* key, size_t key_length, const uint8_t* salt, size_t salt_length);

Description: Performs HKDF-SHA2-384 extract.

Details: This function derives a 48-byte HKDF pseudorandom key from input keying material and an optional salt. The output buffer must be at least see QSCABI_HKDF_SHA2_384_PRK_SIZE bytes.

Parameters: output [out]: The caller-owned output buffer that receives the PRK.; output_length [in]: The size of the output buffer in bytes.; key [in]: The caller-owned input keying-material buffer.; key_length

[in]: The size of the input keying-material buffer in bytes.; salt [in]: The optional caller-owned salt buffer, or NULL when salt_length is zero.; salt_length [in]: The size of the salt buffer in bytes.

Return: Returns see qscabi_status_success on success, otherwise a status code indicating the validation or derivation failure.

qscabi_hkdf_sha2_384_expand

qscabi_status qscabi_hkdf_sha2_384_expand(uint8_t* output, size_t output_length, const uint8_t* key, size_t key_length, const uint8_t* info, size_t info_length);

Description: Performs HKDF-SHA2-384 expand.

Details: This function expands an HKDF-SHA2-384 PRK into caller-requested output using optional context information. The key buffer should contain a PRK produced by see qscabi_hkdf_sha2_384_extract or an equivalent HKDF-SHA2-384 extract step.

Parameters: output [out]: The caller-owned output buffer that receives the derived bytes.; output_length [in]: The number of output bytes to derive.; key [in]: The caller-owned PRK buffer.; key_length [in]: The size of the PRK buffer in bytes.; info [in]: The optional caller-owned context-information buffer, or NULL when info_length is zero.; info_length [in]: The size of the context-information buffer in bytes.

Return: Returns see qscabi_status_success on success, otherwise a status code indicating the validation or derivation failure.

qscabi_hkdf_sha2_512_extract

qscabi_status qscabi_hkdf_sha2_512_extract(uint8_t* output, size_t output_length, const uint8_t* key, size_t key_length, const uint8_t* salt, size_t salt_length);

Description: Performs HKDF-SHA2-512 extract.

Details: This function derives a 64-byte HKDF pseudorandom key from input keying material and an optional salt. The output buffer must be at least see QSCABI_HKDF_SHA2_512_PRK_SIZE bytes.

Parameters: output [out]: The caller-owned output buffer that receives the PRK.; output_length [in]: The size of the output buffer in bytes.; key [in]: The caller-owned input keying-material buffer.; key_length [in]: The size of the input keying-material buffer in bytes.; salt [in]: The optional caller-owned salt buffer, or NULL when salt_length is zero.; salt_length [in]: The size of the salt buffer in bytes.

Return: Returns see qscabi_status_success on success, otherwise a status code indicating the validation or derivation failure.

qscabi_hkdf_sha2_512_expand

qscabi_status qscabi_hkdf_sha2_512_expand(uint8_t* output, size_t output_length, const uint8_t* key, size_t key_length, const uint8_t* info, size_t info_length);

Description: Performs HKDF-SHA2-512 expand.

Details: This function expands an HKDF-SHA2-512 PRK into caller-requested output using optional context information. The key buffer should contain a PRK produced by see qscabi_hkdf_sha2_512_extract or an equivalent HKDF-SHA2-512 extract step.

Parameters: output [out]: The caller-owned output buffer that receives the derived bytes.; output_length [in]: The number of output bytes to derive.; key [in]: The caller-owned PRK buffer.;

key_length [in]: The size of the PRK buffer in bytes.; info [in]: The optional caller-owned context-information buffer, or NULL when info_length is zero.; info_length [in]: The size of the context-information buffer in bytes.

Return: Returns see qscabi_status_success on success, otherwise a status code indicating the validation or derivation failure.

qscabi_pbmac1_derive_key

qscabi_status qscabi_pbmac1_derive_key(uint8_t* output, size_t output_length, qscabi_pbmac1_hash hash, const uint8_t* password, size_t password_length, const uint8_t* salt, size_t salt_length, uint64_t iterations, size_t key_length);

Description: Derives a PBMAC1 key.

Details: This function derives a key from a password and salt using the selected PBMAC1 hash function and iteration count. The requested key_length must be non-zero, must not exceed output_length, and must not exceed see QSCABI_PBMAC1_MAX_KEY_SIZE. The iteration count must be non-zero and must not exceed see QSCABI_PBMAC1_MAX_ITERATIONS.

Parameters: output [out]: The caller-owned output buffer that receives the derived key.; output_length [in]: The size of the output buffer in bytes.; hash [in]: The PBMAC1 hash selector.; password [in]: The caller-owned password buffer.; password_length [in]: The size of the password buffer in bytes.; salt [in]: The caller-owned salt buffer.; salt_length [in]: The size of the salt buffer in bytes.; iterations [in]: The PBMAC1 iteration count.; key_length [in]: The requested derived-key length in bytes.

Return: Returns see qscabi_status_success on success, otherwise a status code indicating the validation or derivation failure.

23.11 qscabi_mac.h — 23 functions

HMAC, KMAC, Poly1305, QMAC and PBMAC1 authentication.

qscabi_hmac_sha2_256_mac_size

size_t qscabi_hmac_sha2_256_mac_size(void);

Description: Gets the HMAC-SHA2-256 authentication-code size.

Return: The HMAC-SHA2-256 MAC size in bytes.

qscabi_hmac_sha2_384_mac_size

size_t qscabi_hmac_sha2_384_mac_size(void);

Description: Gets the HMAC-SHA2-384 authentication-code size.

Return: The HMAC-SHA2-384 MAC size in bytes.

qscabi_hmac_sha2_512_mac_size

size_t qscabi_hmac_sha2_512_mac_size(void);

Description: Gets the HMAC-SHA2-512 authentication-code size.

Return: The HMAC-SHA2-512 MAC size in bytes.

qscabi_poly1305_mac_size

size_t qscabi_poly1305_mac_size(void);

Description: Gets the Poly1305 authentication-code size.

Return: The Poly1305 MAC size in bytes.

qscabi_poly1305_key_size

size_t qscabi_poly1305_key_size(void);

Description: Gets the Poly1305 key size.

Return: The Poly1305 one-time key size in bytes.

qscabi_qmac_mac_size

size_t qscabi_qmac_mac_size(void);

Description: Gets the QMAC authentication-code size.

Return: The QMAC MAC size in bytes.

qscabi_qmac_key_size

size_t qscabi_qmac_key_size(void);

Description: Gets the QMAC key size.

Return: The QMAC key size in bytes.

qscabi_qmac_nonce_size

size_t qscabi_qmac_nonce_size(void);

Description: Gets the QMAC nonce size.

Return: The QMAC nonce size in bytes.

qscabi_pbmac1_mac_size

size_t qscabi_pbmac1_mac_size(qscabi_pbmac1_hash hash);

Description: Gets the PBMAC1 authentication-code size for a hash profile.

Parameters: hash [in]: The PBMAC1 hash profile selector.

Return: The PBMAC1 MAC size in bytes, or zero if the hash profile is not supported.

qscabi_hmac_sha2_256_compute

qscabi_status qscabi_hmac_sha2_256_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length, const uint8_t* key, size_t key_length);

Description: Computes an HMAC-SHA2-256 authentication code.

Parameters: output [out]: The caller-owned output buffer that receives the MAC.; output_length [in]: The size of the output buffer in bytes; must be at least QSCABI_HMAC_SHA2_256_MAC_SIZE.; message [in]: The caller-owned message buffer.; message_length [in]: The message length in bytes.; key [in]: The caller-owned HMAC key buffer.; key_length [in]: The key length in bytes.

Return: qscabi_status_success on success; otherwise a QSCABI status code.

qscabi_hmac_sha2_384_compute

qscabi_status qscabi_hmac_sha2_384_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length, const uint8_t* key, size_t key_length);

Description: Computes an HMAC-SHA2-384 authentication code.

Parameters: output [out]: The caller-owned output buffer that receives the MAC.; output_length [in]: The size of the output buffer in bytes; must be at least QSCABI_HMAC_SHA2_384_MAC_SIZE.; message [in]: The caller-owned message buffer.; message_length [in]: The message length in bytes.; key [in]: The caller-owned HMAC key buffer.; key_length [in]: The key length in bytes.

Return: qscabi_status_success on success; otherwise a QSCABI status code.

qscabi_hmac_sha2_512_compute

qscabi_status qscabi_hmac_sha2_512_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length, const uint8_t* key, size_t key_length);

Description: Computes an HMAC-SHA2-512 authentication code.

Parameters: output [out]: The caller-owned output buffer that receives the MAC.; output_length [in]: The size of the output buffer in bytes; must be at least QSCABI_HMAC_SHA2_512_MAC_SIZE.; message [in]: The caller-owned message buffer.; message_length [in]: The message length in bytes.; key [in]: The caller-owned HMAC key buffer.; key_length [in]: The key length in bytes.

Return: qscabi_status_success on success; otherwise a QSCABI status code.

qscabi_kmac_128_compute

qscabi_status qscabi_kmac_128_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length, const uint8_t* key, size_t key_length, const uint8_t* custom, size_t custom_length);

Description: Computes a KMAC-128 authentication code.

Details: The output length is caller-selected. KMAC customization is supplied through the optional custom buffer.

Parameters: output [out]: The caller-owned output buffer that receives the MAC.; output_length [in]: The requested output length in bytes.; message [in]: The caller-owned message buffer.; message_length [in]: The message length in bytes.; key [in]: The caller-owned KMAC key buffer.; key_length [in]: The key length in bytes.; custom [in]: The optional caller-owned customization string buffer.; custom_length [in]: The customization string length in bytes.

Return: qscabi_status_success on success; otherwise a QSCABI status code.

qscabi_kmac_256_compute

qscabi_status qscabi_kmac_256_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length, const uint8_t* key, size_t key_length, const uint8_t* custom, size_t custom_length);

Description: Computes a KMAC-256 authentication code.

Details: The output length is caller-selected. KMAC customization is supplied through the optional custom buffer.

Parameters: output [out]: The caller-owned output buffer that receives the MAC.; output_length [in]: The requested output length in bytes.; message [in]: The caller-owned message buffer.; message_length [in]: The message length in bytes.; key [in]: The caller-owned KMAC key buffer.; key_length [in]: The key

length in bytes.; custom [in]: The optional caller-owned customization string buffer.; custom_length [in]: The customization string length in bytes.

Return: qscabi_status_success on success; otherwise a QSCABI status code.

qscabi_kmac_512_compute

qscabi_status qscabi_kmac_512_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length, const uint8_t* key, size_t key_length, const uint8_t* custom, size_t custom_length);

Description: Computes a KMAC-512 authentication code.

Details: The output length is caller-selected. KMAC customization is supplied through the optional custom buffer.

Parameters: output [out]: The caller-owned output buffer that receives the MAC.; output_length [in]: The requested output length in bytes.; message [in]: The caller-owned message buffer.; message_length [in]: The message length in bytes.; key [in]: The caller-owned KMAC key buffer.; key_length [in]: The key length in bytes.; custom [in]: The optional caller-owned customization string buffer.; custom_length [in]: The customization string length in bytes.

Return: qscabi_status_success on success; otherwise a QSCABI status code.

qscabi_kmac_xof_128_compute

qscabi_status qscabi_kmac_xof_128_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length, const uint8_t* key, size_t key_length, const uint8_t* custom, size_t custom_length);

Description: Computes KMAC-XOF-128 output.

Details: KMAC-XOF produces caller-selected variable-length output. The customization string may be empty when no application-specific customization is required.

Parameters: output [out]: The caller-owned output buffer that receives the XOF output.; output_length [in]: The requested output length in bytes.; message [in]: The caller-owned message buffer.; message_length [in]: The message length in bytes.; key [in]: The caller-owned KMAC key buffer.; key_length [in]: The key length in bytes.; custom [in]: The optional caller-owned customization string buffer.; custom_length [in]: The customization string length in bytes.

Return: qscabi_status_success on success; otherwise a QSCABI status code.

qscabi_kmac_xof_256_compute

qscabi_status qscabi_kmac_xof_256_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length, const uint8_t* key, size_t key_length, const uint8_t* custom, size_t custom_length);

Description: Computes KMAC-XOF-256 output.

Details: KMAC-XOF produces caller-selected variable-length output. The customization string may be empty when no application-specific customization is required.

Parameters: output [out]: The caller-owned output buffer that receives the XOF output.; output_length [in]: The requested output length in bytes.; message [in]: The caller-owned message buffer.; message_length [in]: The message length in bytes.; key [in]: The caller-owned KMAC key buffer.; key_length [in]: The key length in bytes.; custom [in]: The optional caller-owned customization string buffer.; custom_length [in]: The customization string length in bytes.

Return: qscabi_status_success on success; otherwise a QSCABI status code.

qscabi_kmac_xof_512_compute

qscabi_status qscabi_kmac_xof_512_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length, const uint8_t* key, size_t key_length, const uint8_t* custom, size_t custom_length);

Description: Computes KMAC-XOF-512 output.

Details: KMAC-XOF produces caller-selected variable-length output. The customization string may be empty when no application-specific customization is required.

Parameters: output [out]: The caller-owned output buffer that receives the XOF output.; output_length [in]: The requested output length in bytes.; message [in]: The caller-owned message buffer.; message_length [in]: The message length in bytes.; key [in]: The caller-owned KMAC key buffer.; key_length [in]: The key length in bytes.; custom [in]: The optional caller-owned customization string buffer.; custom_length [in]: The customization string length in bytes.

Return: qscabi_status_success on success; otherwise a QSCABI status code.

qscabi_poly1305_compute

qscabi_status qscabi_poly1305_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length, const uint8_t* key, size_t key_length);

Description: Computes a Poly1305 authentication code.

Parameters: output [out]: The caller-owned output buffer that receives the MAC.; output_length [in]: The size of the output buffer in bytes; must be at least QSCABI_POLY1305_MAC_SIZE.; message [in]: The caller-owned message buffer.; message_length [in]: The message length in bytes.; key [in]: The caller-owned Poly1305 one-time key buffer.; key_length [in]: The key length in bytes; must be QSCABI_POLY1305_KEY_SIZE.

Return: qscabi_status_success on success; otherwise a QSCABI status code.

qscabi_poly1305_verify

qscabi_status qscabi_poly1305_verify(const uint8_t* code, size_t code_length, const uint8_t* message, size_t message_length, const uint8_t* key, size_t key_length);

Description: Verifies a Poly1305 authentication code.

Parameters: code [in]: The caller-owned authentication-code buffer to verify.; code_length [in]: The authentication-code length in bytes; must be QSCABI_POLY1305_MAC_SIZE.; message [in]: The caller-owned message buffer.; message_length [in]: The message length in bytes.; key [in]: The caller-owned Poly1305 one-time key buffer.; key_length [in]: The key length in bytes; must be QSCABI_POLY1305_KEY_SIZE.

Return: qscabi_status_success if the code is valid; qscabi_status_authentication_failure if verification fails; otherwise a QSCABI status code.

qscabi_qmac_compute

qscabi_status qscabi_qmac_compute(uint8_t* output, size_t output_length, const uint8_t* message, size_t message_length, const uint8_t* key, size_t key_length, const uint8_t* nonce, size_t nonce_length, const uint8_t* info, size_t info_length);

Description: Computes a QMAC authentication code.

Details: QMAC uses a caller-supplied key, nonce, and optional information string. The public ABI flattens the QSC parameter model into explicit pointer and length arguments.

Parameters: output [out]: The caller-owned output buffer that receives the MAC.; output_length [in]: The size of the output buffer in bytes; must be at least QSCABI_QMAC_MAC_SIZE.; message [in]: The caller-owned message buffer.; message_length [in]: The message length in bytes.; key [in]: The caller-owned QMAC key buffer.; key_length [in]: The key length in bytes; must be QSCABI_QMAC_KEY_SIZE.; nonce [in]: The caller-owned QMAC nonce buffer.; nonce_length [in]: The nonce length in bytes; must be QSCABI_QMAC_NONCE_SIZE.; info [in]: The optional caller-owned information string buffer.; info_length [in]: The information string length in bytes.

Return: qscabi_status_success on success; otherwise a QSCABI status code.

qscabi_pbmac1_compute

qscabi_status qscabi_pbmac1_compute(uint8_t* output, size_t output_length, qscabi_pbmac1_hash hash, const uint8_t* password, size_t password_length, const uint8_t* salt, size_t salt_length, uint64_t iterations, size_t key_length, const uint8_t* message, size_t message_length);

Description: Computes a PBMAC1 authentication code.

Details: PBMAC1 derives a MAC key from the supplied password, salt, iteration count, key length, and hash profile, then authenticates the supplied message. The password, salt, and derived-key material are not retained after the function returns.

Parameters: output [out]: The caller-owned output buffer that receives the MAC.; output_length [in]: The size of the output buffer in bytes; must match the selected PBMAC1 hash profile.; hash [in]: The PBMAC1 hash profile selector.; password [in]: The caller-owned password buffer.; password_length [in]: The password length in bytes.; salt [in]: The caller-owned salt buffer.; salt_length [in]: The salt length in bytes.; iterations [in]: The PBMAC1 iteration count.; key_length [in]: The derived MAC-key length in bytes.; message [in]: The caller-owned message buffer.; message_length [in]: The message length in bytes.

Return: qscabi_status_success on success; otherwise a QSCABI status code.

qscabi_pbmac1_verify

qscabi_status qscabi_pbmac1_verify(const uint8_t* code, size_t code_length, qscabi_pbmac1_hash hash, const uint8_t* password, size_t password_length, const uint8_t* salt, size_t salt_length, uint64_t iterations, size_t key_length, const uint8_t* message, size_t message_length);

Description: Verifies a PBMAC1 authentication code.

Details: The supplied code is compared against a freshly computed PBMAC1 value for the message and password-derived key parameters.

Parameters: code [in]: The caller-owned authentication-code buffer to verify.; code_length [in]: The authentication-code length in bytes; must match the selected PBMAC1 hash profile.; hash [in]: The PBMAC1 hash profile selector.; password [in]: The caller-owned password buffer.; password_length [in]: The password length in bytes.; salt [in]: The caller-owned salt buffer.; salt_length [in]: The salt length in bytes.; iterations [in]: The PBMAC1 iteration count.; key_length [in]: The derived MAC-key length in bytes.; message [in]: The caller-owned message buffer.; message_length [in]: The message length in bytes.

Return: qscabi_status_success if the code is valid; qscabi_status_authentication_failure if verification fails; otherwise a QSCABI status code.

23.12 qscabi_memory.h — 2 functions

Secret wiping and equality comparison.

qscabi_memory_clear

void qscabi_memory_clear(uint8_t* data, size_t length);

Description: Clears a caller-owned memory region using QSC secure clearing semantics.

Parameters: data: [uint8_t*] Pointer to the caller-owned memory region to clear. The pointer may be null only when length is zero.; length: [size_t] Number of bytes to clear.

Return: void (no direct status result).

Notes: The buffer is overwritten in place. The function does not allocate memory and does not retain the supplied pointer after returning. Passing a null pointer with a non-zero length is ignored by this helper and leaves memory unchanged; public APIs that require output buffers perform their own argument validation. This function clears the memory range supplied by the caller only. It does not clear other copies of the same secret that may exist in managed runtimes, compiler temporaries, logs, or application buffers.

qscabi_memory_are_equal

int qscabi_memory_are_equal(const uint8_t* left, const uint8_t* right, size_t length);

Description: Compares two byte arrays using QSC constant-time comparison semantics.

Parameters: left: [const uint8_t*] Pointer to the first byte array. The pointer may be null only when length is zero.; right: [const uint8_t*] Pointer to the second byte array. The pointer may be null only when length is zero.; length: [size_t] Number of bytes to compare.

Return: Returns 1 when the arrays are equal over length bytes; otherwise returns 0.

Notes: The comparison is intended for secret-bearing byte arrays such as tags, keys, and derived values. The function does not allocate memory and does not retain either input pointer after returning. A zero-length comparison returns 1 when both pointers are non-null and 0 when either pointer is null.

23.13 qscabi_pqc.h — 42 functions

QSC post-quantum KEMs and signatures.

qscabi_mlkem_public_key_size

size_t qscabi_mlkem_public_key_size(void);

Description: Gets the ML-KEM/Kyber public key size.

Return: The public key size in bytes for the configured QSC ML-KEM/Kyber parameter set.

qscabi_mlkem_private_key_size

size_t qscabi_mlkem_private_key_size(void);

Description: Gets the ML-KEM/Kyber private key size.

Return: The private key size in bytes for the configured QSC ML-KEM/Kyber parameter set.

qscabi_mlkem_ciphertext_size

size_t qscabi_mlkem_ciphertext_size(void);

Description: Gets the ML-KEM/Kyber ciphertext size.

Return: The encapsulation ciphertext size in bytes for the configured QSC ML-KEM/Kyber parameter set.

qscabi_mlkem_shared_secret_size

size_t qscabi_mlkem_shared_secret_size(void);

Description: Gets the ML-KEM/Kyber shared-secret size.

Return: The shared-secret size in bytes for the configured QSC ML-KEM/Kyber parameter set.

qscabi_mlkem_generate_keypair

qscabi_status qscabi_mlkem_generate_keypair(uint8_t* publickey, size_t publickey_length, uint8_t* privatekey, size_t privatekey_length);

Description: Generates an ML-KEM/Kyber key pair.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public key output buffer in bytes.; privatekey: The caller-owned output buffer that receives the private key.; privatekey_length: The size of the private key output buffer in bytes.

Return: Returns qscabi_status_success on success; otherwise returns an error status.

qscabi_mlkem_encapsulate

qscabi_status qscabi_mlkem_encapsulate(uint8_t* ciphertext, size_t ciphertext_length, uint8_t* secret, size_t secret_length, const uint8_t* publickey, size_t publickey_length);

Description: Encapsulates a shared secret using an ML-KEM/Kyber public key.

Parameters: ciphertext: The caller-owned output buffer that receives the encapsulation ciphertext.; ciphertext_length: The size of the ciphertext output buffer in bytes.; secret: The caller-owned output buffer that receives the shared secret.; secret_length: The size of the shared-secret output buffer in bytes.; publickey: The caller-owned input buffer containing the recipient public key.; publickey_length: The size of the public key input buffer in bytes.

Return: Returns qscabi_status_success on success; otherwise returns an error status.

qscabi_mlkem_decapsulate

qscabi_status qscabi_mlkem_decapsulate(uint8_t* secret, size_t secret_length, const uint8_t* ciphertext, size_t ciphertext_length, const uint8_t* privatekey, size_t privatekey_length);

Description: Decapsulates an ML-KEM/Kyber ciphertext using a private key.

Parameters: secret: The caller-owned output buffer that receives the recovered shared secret.; secret_length: The size of the shared-secret output buffer in bytes.; ciphertext: The caller-owned input buffer containing the encapsulation ciphertext.; ciphertext_length: The size of the ciphertext input buffer in bytes.; privatekey: The caller-owned input buffer containing the private key.; privatekey_length: The size of the private key input buffer in bytes.

Return: Returns qscabi_status_success on success; otherwise returns an error status.

qscabi_hqc_public_key_size

size_t qscabi_hqc_public_key_size(void);

Description: Gets the HQC public key size.

Return: The public key size in bytes for the configured QSC HQC parameter set.

qscabi_hqc_private_key_size

size_t qscabi_hqc_private_key_size(void);

Description: Gets the HQC private key size.

Return: The private key size in bytes for the configured QSC HQC parameter set.

qscabi_hqc_ciphertext_size

size_t qscabi_hqc_ciphertext_size(void);

Description: Gets the HQC ciphertext size.

Return: The encapsulation ciphertext size in bytes for the configured QSC HQC parameter set.

qscabi_hqc_shared_secret_size

size_t qscabi_hqc_shared_secret_size(void);

Description: Gets the HQC shared-secret size.

Return: The shared-secret size in bytes for the configured QSC HQC parameter set.

qscabi_hqc_generate_keypair

qscabi_status qscabi_hqc_generate_keypair(uint8_t* publickey, size_t publickey_length, uint8_t* privatekey, size_t privatekey_length);

Description: Generates an HQC key pair.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public key output buffer in bytes.; privatekey: The caller-owned output buffer that receives the private key.; privatekey_length: The size of the private key output buffer in bytes.

Return: Returns qscabi_status_success on success; otherwise returns an error status.

qscabi_hqc_encapsulate

qscabi_status qscabi_hqc_encapsulate(uint8_t* ciphertext, size_t ciphertext_length, uint8_t* secret, size_t secret_length, const uint8_t* publickey, size_t publickey_length);

Description: Encapsulates a shared secret using an HQC public key.

Parameters: ciphertext: The caller-owned output buffer that receives the encapsulation ciphertext.; ciphertext_length: The size of the ciphertext output buffer in bytes.; secret: The caller-owned output buffer that receives the shared secret.; secret_length: The size of the shared-secret output buffer in bytes.; publickey: The caller-owned input buffer containing the recipient public key.; publickey_length: The size of the public key input buffer in bytes.

Return: Returns qscabi_status_success on success; otherwise returns an error status.

qscabi_hqc_decapsulate

qscabi_status qscabi_hqc_decapsulate(uint8_t* secret, size_t secret_length, const uint8_t* ciphertext, size_t ciphertext_length, const uint8_t* privatekey, size_t privatekey_length);

Description: Decapsulates an HQC ciphertext using a private key.

Parameters: secret: The caller-owned output buffer that receives the recovered shared secret.; secret_length: The size of the shared-secret output buffer in bytes.; ciphertext: The caller-owned input buffer containing the encapsulation ciphertext.; ciphertext_length: The size of the ciphertext input buffer in bytes.; privatekey: The caller-owned input buffer containing the private key.; privatekey_length: The size of the private key input buffer in bytes.

Return: Returns qscabi_status_success on success; otherwise returns an error status.

qscabi_mceliece_public_key_size

size_t qscabi_mceliece_public_key_size(void);

Description: Gets the Classic McEliece public key size.

Return: The public key size in bytes for the configured QSC Classic McEliece parameter set.

qscabi_mceliece_private_key_size

size_t qscabi_mceliece_private_key_size(void);

Description: Gets the Classic McEliece private key size.

Return: The private key size in bytes for the configured QSC Classic McEliece parameter set.

qscabi_mceliece_ciphertext_size

size_t qscabi_mceliece_ciphertext_size(void);

Description: Gets the Classic McEliece ciphertext size.

Return: The encapsulation ciphertext size in bytes for the configured QSC Classic McEliece parameter set.

qscabi_mceliece_shared_secret_size

size_t qscabi_mceliece_shared_secret_size(void);

Description: Gets the Classic McEliece shared-secret size.

Return: The shared-secret size in bytes for the configured QSC Classic McEliece parameter set.

qscabi_mceliece_generate_keypair

qscabi_status qscabi_mceliece_generate_keypair(uint8_t* publickey, size_t publickey_length, uint8_t* privatekey, size_t privatekey_length);

Description: Generates a Classic McEliece key pair.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public key output buffer in bytes.; privatekey: The caller-owned output buffer that receives the private key.; privatekey_length: The size of the private key output buffer in bytes.

Return: Returns qscabi_status_success on success; otherwise returns an error status.

qscabi_mceliece_encapsulate

qscabi_status qscabi_mceliece_encapsulate(uint8_t* ciphertext, size_t ciphertext_length, uint8_t* secret, size_t secret_length, const uint8_t* publickey, size_t publickey_length);

Description: Encapsulates a shared secret using a Classic McEliece public key.

Parameters: ciphertext: The caller-owned output buffer that receives the encapsulation ciphertext.; ciphertext_length: The size of the ciphertext output buffer in bytes.; secret: The caller-owned output buffer that receives the shared secret.; secret_length: The size of the shared-secret output buffer in bytes.; publickey: The caller-owned input buffer containing the recipient public key.; publickey_length: The size of the public key input buffer in bytes.

Return: Returns qscabi_status_success on success; otherwise returns an error status.

qscabi_mceliece_decapsulate

qscabi_status qscabi_mceliece_decapsulate(uint8_t* secret, size_t secret_length, const uint8_t* ciphertext, size_t ciphertext_length, const uint8_t* privatekey, size_t privatekey_length);

Description: Decapsulates a Classic McEliece ciphertext using a private key.

Parameters: secret: The caller-owned output buffer that receives the recovered shared secret.; secret_length: The size of the shared-secret output buffer in bytes.; ciphertext: The caller-owned input buffer containing the encapsulation ciphertext.; ciphertext_length: The size of the ciphertext input buffer in bytes.; privatekey: The caller-owned input buffer containing the private key.; privatekey_length: The size of the private key input buffer in bytes.

Return: Returns qscabi_status_success on success; otherwise returns an error status.

qscabi_mldsa_public_key_size

size_t qscabi_mldsa_public_key_size(void);

Description: Gets the ML-DSA/Dilithium public key size.

Return: The public key size in bytes for the configured QSC ML-DSA/Dilithium parameter set.

qscabi_mldsa_private_key_size

size_t qscabi_mldsa_private_key_size(void);

Description: Gets the ML-DSA/Dilithium private key size.

Return: The private key size in bytes for the configured QSC ML-DSA/Dilithium parameter set.

qscabi_mldsa_signature_size

size_t qscabi_mldsa_signature_size(void);

Description: Gets the ML-DSA/Dilithium signature size.

Return: The signature size in bytes for the configured QSC ML-DSA/Dilithium parameter set.

qscabi_mldsa_signed_message_size

size_t qscabi_mldsa_signed_message_size(size_t message_length);

Description: Gets the required ML-DSA/Dilithium signed-message buffer size.

Parameters: message_length: The plaintext message length in bytes.

Return: The signed-message buffer size in bytes required for a message of message_length bytes.

qscabi_mldsa_generate_keypair

qscabi_status qscabi_mldsa_generate_keypair(uint8_t* publickey, size_t publickey_length, uint8_t* privatekey, size_t privatekey_length);

Description: Generates an ML-DSA/Dilithium key pair.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public key output buffer in bytes.; privatekey: The caller-owned output buffer that receives the private key.; privatekey_length: The size of the private key output buffer in bytes.

Return: Returns qscabi_status_success on success; otherwise returns an error status.

qscabi_mldsa_sign

qscabi_status qscabi_mldsa_sign(uint8_t* signed_message, size_t signed_message_length, size_t* written, const uint8_t* message, size_t message_length, const uint8_t* privatekey, size_t privatekey_length);

Description: Signs a message using an ML-DSA/Dilithium private key.

Parameters: signed_message: The caller-owned output buffer that receives the signed-message value.; signed_message_length: The size of the signed-message output buffer in bytes.; written: A caller-owned pointer that receives the number of bytes written to signed_message.; message: The caller-owned input buffer containing the message to sign.; message_length: The size of the message input buffer in bytes.; privatekey: The caller-owned input buffer containing the private key.; privatekey_length: The size of the private key input buffer in bytes.

Return: Returns qscabi_status_success on success; otherwise returns an error status.

qscabi_mldsa_verify

qscabi_status qscabi_mldsa_verify(uint8_t* message, size_t message_length, size_t* written, const uint8_t* signed_message, size_t signed_message_length, const uint8_t* publickey, size_t publickey_length);

Description: Verifies an ML-DSA/Dilithium signed message and recovers the message.

Parameters: message: The caller-owned output buffer that receives the verified message.; message_length: The size of the message output buffer in bytes.; written: A caller-owned pointer that receives the number of bytes written to message.; signed_message: The caller-owned input buffer containing the signed-message value.; signed_message_length: The size of the signed-message input buffer in bytes.; publickey: The caller-owned input buffer containing the public key.; publickey_length: The size of the public key input buffer in bytes.

Return: Returns qscabi_status_success on successful verification; otherwise returns an error status.

qscabi_falcon_public_key_size

size_t qscabi_falcon_public_key_size(void);

Description: Gets the Falcon public key size.

Return: The public key size in bytes for the configured QSC Falcon parameter set.

qscabi_falcon_private_key_size

size_t qscabi_falcon_private_key_size(void);

Description: Gets the Falcon private key size.

Return: The private key size in bytes for the configured QSC Falcon parameter set.

qscabi_falcon_signature_size

size_t qscabi_falcon_signature_size(void);

Description: Gets the Falcon signature size.

Return: The signature size in bytes for the configured QSC Falcon parameter set.

qscabi_falcon_signed_message_size

size_t qscabi_falcon_signed_message_size(size_t message_length);

Description: Gets the required Falcon signed-message buffer size.

Parameters: message_length: The plaintext message length in bytes.

Return: The signed-message buffer size in bytes required for a message of message_length bytes.

qscabi_falcon_generate_keypair

qscabi_status qscabi_falcon_generate_keypair(uint8_t* publickey, size_t publickey_length, uint8_t* privatekey, size_t privatekey_length);

Description: Generates a Falcon key pair.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public key output buffer in bytes.; privatekey: The caller-owned output buffer that receives the private key.; privatekey_length: The size of the private key output buffer in bytes.

Return: Returns qscabi_status_success on success; otherwise returns an error status.

qscabi_falcon_sign

qscabi_status qscabi_falcon_sign(uint8_t* signed_message, size_t signed_message_length, size_t* written, const uint8_t* message, size_t message_length, const uint8_t* privatekey, size_t privatekey_length);

Description: Signs a message using a Falcon private key.

Parameters: signed_message: The caller-owned output buffer that receives the signed-message value.; signed_message_length: The size of the signed-message output buffer in bytes.; written: A caller-owned pointer that receives the number of bytes written to signed_message.; message: The caller-owned input buffer containing the message to sign.; message_length: The size of the message input buffer in bytes.; privatekey: The caller-owned input buffer containing the private key.; privatekey_length: The size of the private key input buffer in bytes.

Return: Returns qscabi_status_success on success; otherwise returns an error status.

qscabi_falcon_verify

qscabi_status qscabi_falcon_verify(uint8_t* message, size_t message_length, size_t* written, const uint8_t* signed_message, size_t signed_message_length, const uint8_t* publickey, size_t publickey_length);

Description: Verifies a Falcon signed message and recovers the message.

Parameters: message: The caller-owned output buffer that receives the verified message.; message_length: The size of the message output buffer in bytes.; written: A caller-owned pointer that receives the number of bytes written to message.; signed_message: The caller-owned input buffer containing the signed-message value.; signed_message_length: The size of the signed-message input buffer in bytes.; publickey: The caller-owned input buffer containing the public key.; publickey_length: The size of the public key input buffer in bytes.

Return: Returns qscabi_status_success on successful verification; otherwise returns an error status.

qscabi_slhdsa_public_key_size

size_t qscabi_slhdsa_public_key_size(void);

Description: Gets the SLH-DSA/SPHINCS+ public key size.

Return: The public key size in bytes for the configured QSC SLH-DSA/SPHINCS+ parameter set.

qscabi_slhdsa_private_key_size

size_t qscabi_slhdsa_private_key_size(void);

Description: Gets the SLH-DSA/SPHINCS+ private key size.

Return: The private key size in bytes for the configured QSC SLH-DSA/SPHINCS+ parameter set.

qscabi_slhdsa_signature_size

size_t qscabi_slhdsa_signature_size(void);

Description: Gets the SLH-DSA/SPHINCS+ signature size.

Return: The signature size in bytes for the configured QSC SLH-DSA/SPHINCS+ parameter set.

qscabi_slhdsa_signed_message_size

size_t qscabi_slhdsa_signed_message_size(size_t message_length);

Description: Gets the required SLH-DSA/SPHINCS+ signed-message buffer size.

Parameters: message_length: The plaintext message length in bytes.

Return: The signed-message buffer size in bytes required for a message of message_length bytes.

qscabi_slhdsa_generate_keypair

qscabi_status qscabi_slhdsa_generate_keypair(uint8_t* publickey, size_t publickey_length, uint8_t* privatekey, size_t privatekey_length);

Description: Generates an SLH-DSA/SPHINCS+ key pair.

Parameters: publickey: The caller-owned output buffer that receives the public key.; publickey_length: The size of the public key output buffer in bytes.; privatekey: The caller-owned output buffer that receives the private key.; privatekey_length: The size of the private key output buffer in bytes.

Return: Returns qscabi_status_success on success; otherwise returns an error status.

qscabi_slhdsa_sign

qscabi_status qscabi_slhdsa_sign(uint8_t* signed_message, size_t signed_message_length, size_t* written, const uint8_t* message, size_t message_length, const uint8_t* privatekey, size_t privatekey_length);

Description: Signs a message using an SLH-DSA/SPHINCS+ private key.

Parameters: signed_message: The caller-owned output buffer that receives the signed-message value.; signed_message_length: The size of the signed-message output buffer in bytes.; written: A caller-owned pointer that receives the number of bytes written to signed_message.; message: The caller-owned input buffer containing the message to sign.; message_length: The size of the message input buffer in bytes.; privatekey: The caller-owned input buffer containing the private key.; privatekey_length: The size of the private key input buffer in bytes.

Return: Returns qscabi_status_success on success; otherwise returns an error status.

qscabi_slhdsa_verify

qscabi_status qscabi_slhdsa_verify(uint8_t* message, size_t message_length, size_t* written, const uint8_t* signed_message, size_t signed_message_length, const uint8_t* publickey, size_t publickey_length);

Description: Verifies an SLH-DSA/SPHINCS+ signed message and recovers the message.

Parameters: message: The caller-owned output buffer that receives the verified message.; message_length: The size of the message output buffer in bytes.; written: A caller-owned pointer that receives the number of bytes written to message.; signed_message: The caller-owned input buffer containing the signed-message value.; signed_message_length: The size of the signed-message input buffer in bytes.; publickey: The caller-owned input buffer containing the public key.; publickey_length: The size of the public key input buffer in bytes.

Return: Returns qscabi_status_success on successful verification; otherwise returns an error status.

23.14 qscabi_status.h — 1 functions

Stable status strings for errors and diagnostics.

qscabi_status_string

const char* qscabi_status_string(qscabi_status status);

Description: Returns a stable constant string for a QSCABI status value.

Parameters: status: [qscabi_status] The status code to translate.

Return: Returns a null-terminated static string. The caller must not free or modify the returned pointer.

Notes: The returned string is a diagnostic label intended for logs, test harness output, binding diagnostics, and human-readable exception messages. Unknown or out-of-range values return the string "unknown".

23.15 qscabi_stream.h — 18 functions

Opaque digest, XOF, MAC and CTR streaming states.

qscabi_digest_size

size_t qscabi_digest_size(qscabi_digest_algorithm algorithm);

Description: Query exact digest length in bytes.

Parameters: algorithm: Digest ID.

Return: Output size, zero for unsupported IDs.

qscabi_digest_create

qscabi_status qscabi_digest_create(qscabi_digest_algorithm algorithm, qscabi_digest_state** state);

Description: Allocate an active incremental digest context.

Parameters: algorithm: Digest ID.; state [out]: Receives owned opaque handle.

Return: Success or error; destroy to erase and release the handle.

qscabi_digest_update

qscabi_status qscabi_digest_update(qscabi_digest_state* state, const uint8_t* data, size_t length);

Description: Append digest input, including optional empty input.

Parameters: state: Initialized handle.; data: Borrowed input.; length: Number of input bytes.

Return: Success or invalid-state/argument.

qscabi_digest_finalize

qscabi_status qscabi_digest_finalize(qscabi_digest_state* state, uint8_t* digest, size_t digest_length);

Description: Finalize digest once, writing exact-length output; reject repeated calls.

Parameters: state: Active context.; digest [out]: Caller output.; digest_length: Capacity, must be at least qscabi_digest_size.

Return: Status; no digest result after failed preflight or repeated finalize.

qscabi_digest_destroy

void qscabi_digest_destroy(qscabi_digest_state* state);

Description: Securely erase and free digest handle; null is safe.

Parameters: state: Handle returned by create. Do not reuse after this call.

Return: void (no direct status result).

qscabi_xof_create

qscabi_status qscabi_xof_create(qscabi_xof_algorithm algorithm, qscabi_xof_state** state);

Description: Initialize a SHAKE sponge with no absorbed bytes.

Parameters: algorithm: XOF strength.; state [out]: New owned handle.

Return: Status; state must be destroyed by this library.

qscabi_xof_absorb

qscabi_status qscabi_xof_absorb(qscabi_xof_state* state, const uint8_t* data, size_t length);

Description: Absorb more bytes before finalize.

Parameters: state: Active sponge.; data: Borrowed data (null only for zero length).; length: Byte count.

Return: Success or invalid-state/argument.

qscabi_xof_finalize

qscabi_status qscabi_xof_finalize(qscabi_xof_state* state);

Description: Transition the sponge to squeezing; may be called only once.

Parameters: state: Absorbing sponge.

Return: Success or invalid-state.

qscabi_xof_squeeze

qscabi_status qscabi_xof_squeeze(qscabi_xof_state* state, uint8_t* output, size_t length);

Description: Squeeze bytes repeatedly, preserving the sponge position.

Parameters: state: Finalized sponge.; output [out]: Caller-owned output.; length: Output bytes, may be zero.

Return: Status.

qscabi_xof_destroy

void qscabi_xof_destroy(qscabi_xof_state* state);

Description: Securely erase and free XOF handle; null is safe.

Parameters: state: Owned handle, invalid after release.

Return: void (no direct status result).

qscabi_cshake_compute

qscabi_status qscabi_cshake_compute(qscabi_xof_algorithm algorithm, const uint8_t* key, size_t key_length, const uint8_t* name, size_t name_length, const uint8_t* custom, size_t custom_length, uint8_t* output, size_t output_length);

Description: Calculate a customized cSHAKE output without retaining source bytes.

Parameters: algorithm: Strength (128, 256, or 512).; key: Borrowed input.; key_length: Input size.; name: Borrowed function name.; name_length: Name size.; custom: Borrowed customization string.; custom_length: Custom size.; output [out]: Caller output.; output_length: Required output bytes; nonzero.

Return: Success or input validation error; independent calls are thread-safe.

qscabi_stream_mac_create

qscabi_status qscabi_stream_mac_create(qscabi_stream_mac_algorithm algorithm, const uint8_t* key, size_t key_length, const uint8_t* custom, size_t custom_length, qscabi_stream_mac_state** state);

Description: Create an incremental MAC with caller-owned secret key.

Parameters: algorithm: HMAC/KMAC selection.; key: Nonempty key, borrowed only during call.; key_length: Key bytes.; custom: KMAC personalization; empty for HMAC.; custom_length: Personalization bytes; HMAC requires zero.; state [out]: Owned handle.

Return: Status, including allocation failure.

qscabi_stream_mac_update

qscabi_status qscabi_stream_mac_update(qscabi_stream_mac_state* state, const uint8_t* data, size_t length);

Description: Absorb MAC message bytes.

Parameters: state: Active MAC handle.; data: Message.; length: Message bytes, may be zero.

Return: Status; reject repeated use after finalize.

qscabi_stream_mac_finalize

qscabi_status qscabi_stream_mac_finalize(qscabi_stream_mac_state* state, uint8_t* output, size_t length);

Description: Produce a final MAC and erase native key state.

Parameters: state: Active MAC handle.; output [out]: Caller-owned tag.; length: HMAC requires exact digest bytes, KMAC permits nonzero length.

Return: Status; duplicate finalize rejected.

qscabi_stream_mac_destroy

void qscabi_stream_mac_destroy(qscabi_stream_mac_state* state);

Description: Securely erase and release opaque MAC state.

Parameters: state: Owned handle, null allowed.

Return: void (no direct status result).

qscabi_ctr_create

qscabi_status qscabi_ctr_create(qscabi_ctr_algorithm algorithm, const uint8_t* key, size_t key_length, const uint8_t* nonce, size_t nonce_length, qscabi_ctr_state** state);

Description: Create AES CTR state; never reuse a counter under the same key.

Parameters: algorithm: AES-128/256 CTR endian mode.; key: Exact-sized key.; key_length: Key bytes.; nonce: Fresh 16-byte counter, copied into state.; nonce_length: Exactly 16.; state [out]: Owned handle.

Return: Status; application must provide unique key/counter pair.

qscabi_ctr_transform

qscabi_status qscabi_ctr_transform(qscabi_ctr_state* state, const uint8_t* input, size_t length, uint8_t* output, size_t output_length);

Description: Transform arbitrary input; partial final blocks end the stream.

Details: Each update is block-aligned except the final call, which may end with a partial block. Updating again after a partial block is rejected to prevent keystream reuse. CTR provides NO authentication or tamper protection.

Parameters: state: Active AES CTR.; input: Borrowed input bytes.; length: Input/output bytes; may be zero.; output [out]: Caller result.; output_length: Destination size, at least length.

Return: Status. Source and destination must not overlap.

qscabi_ctr_destroy

void qscabi_ctr_destroy(qscabi_ctr_state* state);

Description: Securely erase AES key schedule and counter and free handle.

Parameters: state: Owned handle; null is safe.

Return: void (no direct status result).

23.16 qscabi_symmetric.h — 27 functions

AES, ChaCha, CSX, RCS and AEAD one-shot operations.

qscabi_aes_block_size

size_t qscabi_aes_block_size(void);

Description: Get the AES block size.

Return: The AES block size in bytes.

qscabi_aes128_key_size

size_t qscabi_aes128_key_size(void);

Description: Get the AES-128 key size.

Return: The AES-128 key size in bytes.

qscabi_aes256_key_size

size_t qscabi_aes256_key_size(void);

Description: Get the AES-256 key size.

Return: The AES-256 key size in bytes.

qscabi_aes_gcm_nonce_size

size_t qscabi_aes_gcm_nonce_size(void);

Description: Get the recommended AES-GCM nonce size.

Return: The recommended AES-GCM nonce size in bytes.

qscabi_aes_gcm_max_nonce_size

size_t qscabi_aes_gcm_max_nonce_size(void);

Description: Get the maximum AES-GCM nonce size accepted by the ABI.

Return: The maximum AES-GCM nonce size in bytes.

qscabi_aes_gcm_tag_size

size_t qscabi_aes_gcm_tag_size(void);

Description: Get the AES-GCM authentication tag size.

Return: The AES-GCM authentication tag size in bytes.

qscabi_chacha20_key_size

size_t qscabi_chacha20_key_size(void);

Description: Get the ChaCha20 key size.

Return: The ChaCha20 key size in bytes.

qscabi_chacha20_nonce_size

size_t qscabi_chacha20_nonce_size(void);

Description: Get the ChaCha20 nonce size.

Return: The ChaCha20 nonce size in bytes.

qscabi_chacha20_poly1305_tag_size

size_t qscabi_chacha20_poly1305_tag_size(void);

Description: Get the ChaCha20-Poly1305 authentication tag size.

Return: The ChaCha20-Poly1305 authentication tag size in bytes.

qscabi_csx_key_size

size_t qscabi_csx_key_size(void);

Description: Get the CSX key size.

Return: The CSX key size in bytes.

qscabi_csx_nonce_size

size_t qscabi_csx_nonce_size(void);

Description: Get the CSX nonce size.

Return: The CSX nonce size in bytes.

qscabi_csx_tag_size

size_t qscabi_csx_tag_size(void);

Description: Get the CSX authentication tag size.

Return: The CSX authentication tag size in bytes.

qscabi_rcs_key_size

size_t qscabi_rcs_key_size(qscabi_rcs_mode mode);

Description: Get the RCS key size for a selected mode.

Parameters: mode: The RCS parameter-set selector.

Return: The RCS key size in bytes, or zero if mode is invalid.

qscabi_rcs_nonce_size

size_t qscabi_rcs_nonce_size(void);

Description: Get the RCS nonce size.

Return: The RCS nonce size in bytes.

qscabi_rcs_tag_size

size_t qscabi_rcs_tag_size(qscabi_rcs_mode mode);

Description: Get the RCS authentication tag size for a selected mode.

Parameters: mode: The RCS parameter-set selector.

Return: The RCS authentication tag size in bytes, or zero if mode is invalid.

qscabi_aes256_ctr_transform

qscabi_status qscabi_aes256_ctr_transform(uint8_t* output, size_t output_length, const uint8_t* input, size_t input_length, const uint8_t* key, size_t key_length, const uint8_t* nonce, size_t nonce_length, qscabi_aes_ctr_counter counter_mode);

Description: Transform data using AES-256 in CTR mode.

Details: AES-CTR is a stream transformation. Encryption and decryption use the same operation. The output buffer must contain at least input_length bytes. The key buffer must contain exactly QSCABI_AES256_KEY_SIZE bytes and the nonce buffer must contain exactly QSCABI_AES_BLOCK_SIZE bytes.

Parameters: output [out]: Pointer to the caller-owned output buffer.; output_length [in]: Size of the output buffer in bytes.; input [in]: Pointer to the caller-owned input buffer. This pointer may be null only when input_length is zero.; input_length [in]: Number of bytes to transform.; key [in]: Pointer to the AES-256 key.; key_length [in]: Length of key in bytes.; nonce [in]: Pointer to the 16-byte counter block.; nonce_length [in]: Length of nonce in bytes.; counter_mode [in]: Counter byte-order mode.

Return: Returns qscabi_status_success on success, or a non-success status if an argument, size, or mode selector is invalid.

Notes: Reusing the same AES-CTR key and counter-block sequence compromises confidentiality.

qscabi_aes128_gcm_encrypt

qscabi_status qscabi_aes128_gcm_encrypt(uint8_t* output, size_t output_length, size_t* written, const uint8_t* plaintext, size_t plaintext_length, const uint8_t* key, size_t key_length, const uint8_t* nonce, size_t nonce_length, const uint8_t* aad, size_t aad_length);

Description: Encrypt plaintext using AES-128-GCM authenticated encryption.

Details: The output buffer receives ciphertext followed by the AES-GCM authentication tag. The buffer must contain at least plaintext_length + QSCABI_AES_GCM_TAG_SIZE bytes. Optional additional authenticated data contributes to authentication but is not encrypted or written to the output.

Parameters: output [out]: Pointer to the caller-owned ciphertext-and-tag buffer.; output_length [in]: Size of the output buffer in bytes.; written [out]: Receives the number of bytes written to output.; plaintext [in]: Pointer to the plaintext buffer. This pointer may be null only when plaintext_length is zero.; plaintext_length [in]: Number of plaintext bytes to encrypt.; key [in]: Pointer to the AES-128 key.; key_length [in]: Length of key in bytes.; nonce [in]: Pointer to the AES-GCM nonce.; nonce_length [in]: Length of nonce in bytes; must be nonzero and no larger than QSCABI_AES_GCM_MAX_NONCE_SIZE.; aad [in]: Pointer to optional additional authenticated data. This pointer may be null only when aad_length is zero.; aad_length [in]: Length of aad in bytes.

Return: Returns qscabi_status_success on success, or a non-success status if validation fails.

Notes: The same AES-GCM key and nonce pair must never be reused.

qscabi_aes128_gcm_decrypt

qscabi_status qscabi_aes128_gcm_decrypt(uint8_t* output, size_t output_length, size_t* written, const uint8_t* ciphertext, size_t ciphertext_length, const uint8_t* key, size_t key_length, const uint8_t* nonce, size_t nonce_length, const uint8_t* aad, size_t aad_length);

Description: Decrypt ciphertext using AES-128-GCM authenticated encryption.

Details: The ciphertext buffer must contain ciphertext followed by a QSCABI_AES_GCM_TAG_SIZE byte authentication tag. On authentication failure, the function returns qscabi_status_authentication_failure, stores zero in written, and callers must discard the output buffer contents.

Parameters: output [out]: Pointer to the caller-owned plaintext buffer.; output_length [in]: Size of the output buffer in bytes.; written [out]: Receives the number of plaintext bytes written to output.; ciphertext [in]: Pointer to the ciphertext-and-tag input buffer.; ciphertext_length [in]: Length of ciphertext in bytes.; key [in]: Pointer to the AES-128 key.; key_length [in]: Length of key in bytes.; nonce [in]: Pointer to the AES-GCM nonce.; nonce_length [in]: Length of nonce in bytes.; aad [in]: Pointer to optional additional authenticated data. This pointer may be null only when aad_length is zero.; aad_length [in]: Length of aad in bytes.

Return: Returns qscabi_status_success on successful authentication and decryption, or a non-success status on validation or authentication failure.

qscabi_aes256_gcm_encrypt

qscabi_status qscabi_aes256_gcm_encrypt(uint8_t* output, size_t output_length, size_t* written, const uint8_t* plaintext, size_t plaintext_length, const uint8_t* key, size_t key_length, const uint8_t* nonce, size_t nonce_length, const uint8_t* aad, size_t aad_length);

Description: Encrypt plaintext using AES-256-GCM authenticated encryption.

Details: This function is the AES-256 key-size variant of qscabi_aes128_gcm_encrypt.

Parameters: output [out]: Pointer to the caller-owned ciphertext-and-tag buffer.; output_length [in]: Size of the output buffer in bytes.; written [out]: Receives the number of bytes written to output.; plaintext [in]: Pointer to the plaintext buffer. This pointer may be null only when plaintext_length is zero.; plaintext_length [in]: Number of plaintext bytes to encrypt.; key [in]: Pointer to the AES-256 key.; key_length [in]: Length of key in bytes.; nonce [in]: Pointer to the AES-GCM nonce.; nonce_length [in]: Length of nonce in bytes.; aad [in]: Pointer to optional additional authenticated data. This pointer may be null only when aad_length is zero.; aad_length [in]: Length of aad in bytes.

Return: Returns qscabi_status_success on success, or a non-success status if validation fails.

Notes: The same AES-GCM key and nonce pair must never be reused.

qscabi_aes256_gcm_decrypt

qscabi_status qscabi_aes256_gcm_decrypt(uint8_t* output, size_t output_length, size_t* written, const uint8_t* ciphertext, size_t ciphertext_length, const uint8_t* key, size_t key_length, const uint8_t* nonce, size_t nonce_length, const uint8_t* aad, size_t aad_length);

Description: Decrypt ciphertext using AES-256-GCM authenticated encryption.

Details: This function is the AES-256 key-size variant of qscabi_aes128_gcm_decrypt.

Parameters: output [out]: Pointer to the caller-owned plaintext buffer.; output_length [in]: Size of the output buffer in bytes.; written [out]: Receives the number of plaintext bytes written to output.; ciphertext [in]: Pointer to the ciphertext-and-tag input buffer.; ciphertext_length [in]: Length of ciphertext in bytes.; key [in]: Pointer to the AES-256 key.; key_length [in]: Length of key in bytes.; nonce [in]: Pointer to the AES-GCM nonce.; nonce_length [in]: Length of nonce in bytes.; aad [in]: Pointer to optional additional authenticated data. This pointer may be null only when aad_length is zero.; aad_length [in]: Length of aad in bytes.

Return: Returns qscabi_status_success on successful authentication and decryption, or a non-success status on validation or authentication failure.

qscabi_chacha20_transform

qscabi_status qscabi_chacha20_transform(uint8_t* output, size_t output_length, const uint8_t* input, size_t input_length, const uint8_t* key, size_t key_length, const uint8_t* nonce, size_t nonce_length);

Description: Transform data using ChaCha20.

Details: ChaCha20 is a stream transformation. Encryption and decryption use the same operation. The output buffer must contain at least input_length bytes. The key must be QSCABI_CHACHA20_KEY_SIZE bytes and the nonce must be QSCABI_CHACHA20_NONCE_SIZE bytes.

Parameters: output [out]: Pointer to the caller-owned output buffer.; output_length [in]: Size of the output buffer in bytes.; input [in]: Pointer to the caller-owned input buffer. This pointer may be null only when input_length is zero.; input_length [in]: Number of bytes to transform.; key [in]: Pointer to the

ChaCha20 key.; key_length [in]: Length of key in bytes.; nonce [in]: Pointer to the ChaCha20 nonce.; nonce_length [in]: Length of nonce in bytes.

Return: Returns qscabi_status_success on success, or a non-success status if validation fails.

Notes: Reusing the same ChaCha20 key and nonce pair compromises confidentiality.

qscabi_chacha20_poly1305_encrypt

qscabi_status qscabi_chacha20_poly1305_encrypt(uint8_t* output, size_t output_length, size_t* written, const uint8_t* plaintext, size_t plaintext_length, const uint8_t* key, size_t key_length, const uint8_t* nonce, size_t nonce_length, const uint8_t* aad, size_t aad_length);

Description: Encrypt plaintext using ChaCha20-Poly1305 authenticated encryption.

Details: The output buffer receives ciphertext followed by a QSCABI_CHACHA20_POLY1305_TAG_SIZE byte authentication tag.

Parameters: output [out]: Pointer to the caller-owned ciphertext-and-tag buffer.; output_length [in]: Size of the output buffer in bytes.; written [out]: Receives the number of bytes written to output.; plaintext [in]: Pointer to the plaintext buffer. This pointer may be null only when plaintext_length is zero.; plaintext_length [in]: Number of plaintext bytes to encrypt.; key [in]: Pointer to the ChaCha20-Poly1305 key.; key_length [in]: Length of key in bytes.; nonce [in]: Pointer to the ChaCha20-Poly1305 nonce.; nonce_length [in]: Length of nonce in bytes.; aad [in]: Pointer to optional additional authenticated data. This pointer may be null only when aad_length is zero.; aad_length [in]: Length of aad in bytes.

Return: Returns qscabi_status_success on success, or a non-success status if validation fails.

Notes: The same ChaCha20-Poly1305 key and nonce pair must never be reused.

qscabi_chacha20_poly1305_decrypt

qscabi_status qscabi_chacha20_poly1305_decrypt(uint8_t* output, size_t output_length, size_t* written, const uint8_t* ciphertext, size_t ciphertext_length, const uint8_t* key, size_t key_length, const uint8_t* nonce, size_t nonce_length, const uint8_t* aad, size_t aad_length);

Description: Decrypt ciphertext using ChaCha20-Poly1305 authenticated encryption.

Details: The ciphertext buffer must contain ciphertext followed by a QSCABI_CHACHA20_POLY1305_TAG_SIZE byte authentication tag. Authentication failure is reported as qscabi_status_authentication_failure.

Parameters: output [out]: Pointer to the caller-owned plaintext buffer.; output_length [in]: Size of the output buffer in bytes.; written [out]: Receives the number of plaintext bytes written to output.; ciphertext [in]: Pointer to the ciphertext-and-tag input buffer.; ciphertext_length [in]: Length of ciphertext in bytes.; key [in]: Pointer to the ChaCha20-Poly1305 key.; key_length [in]: Length of key in bytes.; nonce [in]: Pointer to the ChaCha20-Poly1305 nonce.; nonce_length [in]: Length of nonce in bytes.; aad [in]: Pointer to optional additional authenticated data. This pointer may be null only when aad_length is zero.; aad_length [in]: Length of aad in bytes.

Return: Returns qscabi_status_success on successful authentication and decryption, or a non-success status on validation or authentication failure.

qscabi_csx_encrypt

qscabi_status qscabi_csx_encrypt(uint8_t* output, size_t output_length, size_t* written, const uint8_t* plaintext, size_t plaintext_length, const uint8_t* key, size_t key_length, const uint8_t* nonce, size_t nonce_length, const uint8_t* info, size_t info_length, const uint8_t* aad, size_t aad_length);

Description: Encrypt plaintext using CSX authenticated encryption.

Details: The output buffer receives CSX ciphertext followed by a QSCABI_CSX_TAG_SIZE byte authentication tag. The info parameter supplies optional customization or domain-separation data. The aad parameter supplies additional authenticated data that is authenticated but not encrypted.

Parameters: output [out]: Pointer to the caller-owned ciphertext-and-tag buffer.; output_length [in]: Size of the output buffer in bytes.; written [out]: Receives the number of bytes written to output.; plaintext [in]: Pointer to the plaintext buffer. This pointer may be null only when plaintext_length is zero.; plaintext_length [in]: Number of plaintext bytes to encrypt.; key [in]: Pointer to the CSX key.; key_length [in]: Length of key in bytes.; nonce [in]: Pointer to the CSX nonce.; nonce_length [in]: Length of nonce in bytes.; info [in]: Pointer to optional customization data. This pointer may be null only when info_length is zero.; info_length [in]: Length of info in bytes.; aad [in]: Pointer to optional additional authenticated data. This pointer may be null only when aad_length is zero.; aad_length [in]: Length of aad in bytes.

Return: Returns qscabi_status_success on success, or a non-success status if validation fails.

Notes: Reusing a CSX key and nonce pair compromises security.

qscabi_csx_decrypt

qscabi_status qscabi_csx_decrypt(uint8_t* output, size_t output_length, size_t* written, const uint8_t* ciphertext, size_t ciphertext_length, const uint8_t* key, size_t key_length, const uint8_t* nonce, size_t nonce_length, const uint8_t* info, size_t info_length, const uint8_t* aad, size_t aad_length);

Description: Decrypt ciphertext using CSX authenticated encryption.

Details: The ciphertext buffer must contain CSX ciphertext followed by a QSCABI_CSX_TAG_SIZE byte authentication tag. Authentication failure is reported as qscabi_status_authentication_failure.

Parameters: output [out]: Pointer to the caller-owned plaintext buffer.; output_length [in]: Size of the output buffer in bytes.; written [out]: Receives the number of plaintext bytes written to output.; ciphertext [in]: Pointer to the ciphertext-and-tag input buffer.; ciphertext_length [in]: Length of ciphertext in bytes.; key [in]: Pointer to the CSX key.; key_length [in]: Length of key in bytes.; nonce [in]: Pointer to the CSX nonce.; nonce_length [in]: Length of nonce in bytes.; info [in]: Pointer to optional customization data. This pointer may be null only when info_length is zero.; info_length [in]: Length of info in bytes.; aad [in]: Pointer to optional additional authenticated data. This pointer may be null only when aad_length is zero.; aad_length [in]: Length of aad in bytes.

Return: Returns qscabi_status_success on successful authentication and decryption, or a non-success status on validation or authentication failure.

qscabi_rcs_encrypt

qscabi_status qscabi_rcs_encrypt(uint8_t* output, size_t output_length, size_t* written, qscabi_rcs_mode mode, const uint8_t* plaintext, size_t plaintext_length, const uint8_t* key, size_t key_length, const uint8_t* nonce, size_t nonce_length, const uint8_t* info, size_t info_length, const uint8_t* aad, size_t aad_length);

Description: Encrypt plaintext using RCS authenticated encryption.

Details: The output buffer receives RCS ciphertext followed by the authentication tag for mode. Use qscabi_rcs_key_size and qscabi_rcs_tag_size to query the required key and tag sizes for the selected mode.

Parameters: output [out]: Pointer to the caller-owned ciphertext-and-tag buffer.; output_length [in]: Size of the output buffer in bytes.; written [out]: Receives the number of bytes written to output.; mode [in]: RCS parameter-set selector.; plaintext [in]: Pointer to the plaintext buffer. This pointer may be null only when plaintext_length is zero.; plaintext_length [in]: Number of plaintext bytes to encrypt.; key [in]: Pointer to the RCS key.; key_length [in]: Length of key in bytes.; nonce [in]: Pointer to the RCS nonce.; nonce_length [in]: Length of nonce in bytes.; info [in]: Pointer to optional customization data. This pointer may be null only when info_length is zero.; info_length [in]: Length of info in bytes.; aad [in]: Pointer to optional additional authenticated data. This pointer may be null only when aad_length is zero.; aad_length [in]: Length of aad in bytes.

Return: Returns qscabi_status_success on success, or a non-success status if validation fails.

Notes: Reusing an RCS key and nonce pair compromises security.

qscabi_rcs_decrypt

qscabi_status qscabi_rcs_decrypt(uint8_t* output, size_t output_length, size_t* written, qscabi_rcs_mode mode, const uint8_t* ciphertext, size_t ciphertext_length, const uint8_t* key, size_t key_length, const uint8_t* nonce, size_t nonce_length, const uint8_t* info, size_t info_length, const uint8_t* aad, size_t aad_length);

Description: Decrypt ciphertext using RCS authenticated encryption.

Details: The ciphertext buffer must contain RCS ciphertext followed by the authentication tag for mode. Authentication failure is reported as qscabi_status_authentication_failure.

Parameters: output [out]: Pointer to the caller-owned plaintext buffer.; output_length [in]: Size of the output buffer in bytes.; written [out]: Receives the number of plaintext bytes written to output.; mode [in]: RCS parameter-set selector.; ciphertext [in]: Pointer to the ciphertext-and-tag input buffer.; ciphertext_length [in]: Length of ciphertext in bytes.; key [in]: Pointer to the RCS key.; key_length [in]: Length of key in bytes.; nonce [in]: Pointer to the RCS nonce.; nonce_length [in]: Length of nonce in bytes.; info [in]: Pointer to optional customization data. This pointer may be null only when info_length is zero.; info_length [in]: Length of info in bytes.; aad [in]: Pointer to optional additional authenticated data. This pointer may be null only when aad_length is zero.; aad_length [in]: Length of aad in bytes.

Return: Returns qscabi_status_success on successful authentication and decryption, or a non-success status on validation or authentication failure.

23.17 qscabi_tls.h — 49 functions

TLS context, connection, listener and server operations.

qscabi_tls_status_string

const char* qscabi_tls_status_string(qscabi_status status);

Description: Gets a TLS-related status string.

Parameters: status: The QSCABI status code to describe.

Return: A static, null-terminated status string.

qscabi_tls_socket_options_default

void qscabi_tls_socket_options_default(qscabi_tls_socket_options* options);

Description: Initializes TLS socket options with default values.

Parameters: options: The caller-owned options structure to initialize.

Return: void (no direct status result).

qscabi_tls_ticket_policy_default

void qscabi_tls_ticket_policy_default(qscabi_tls_ticket_policy* policy);

Description: Initializes a TLS session-ticket policy with default values.

Parameters: policy: The caller-owned ticket-policy structure to initialize.

Return: void (no direct status result).

qscabi_tls_context_create

qscabi_status qscabi_tls_context_create(qscabi_tls_context** context);

Description: Creates a TLS context handle.

Parameters: context: The address that receives the created TLS context handle.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_context_destroy

void qscabi_tls_context_destroy(qscabi_tls_context* context);

Description: Destroys a TLS context handle.

Parameters: context: The TLS context handle to destroy, or NULL.

Return: void (no direct status result).

qscabi_tls_context_set_policy

qscabi_status qscabi_tls_context_set_policy(qscabi_tls_context* context, qscabi_tls_policy policy);

Description: Applies a predefined TLS policy to a context.

Parameters: context: The TLS context handle.; policy: The policy selector to apply.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_context_set_socket_options

qscabi_status qscabi_tls_context_set_socket_options(qscabi_tls_context* context, const qscabi_tls_socket_options* options);

Description: Sets TLS context socket options.

Parameters: context: The TLS context handle.; options: The caller-owned socket options to copy into the context.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_context_set_alpn_protocols

qscabi_status qscabi_tls_context_set_alpn_protocols(qscabi_tls_context* context, const char* const* protocols, size_t protocol_count, int required);

Description: Sets the ALPN protocol list for a TLS context.

Parameters: context: The TLS context handle.; protocols: The caller-owned array of null-terminated protocol strings.; protocol_count: The number of protocol entries in the array.; required: Non-zero if ALPN selection is required.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_context_clear_alpn_protocols

qscabi_status qscabi_tls_context_clear_alpn_protocols(qscabi_tls_context* context);

Description: Clears the ALPN protocol list from a TLS context.

Parameters: context: The TLS context handle.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_context_set_sni_required

qscabi_status qscabi_tls_context_set_sni_required(qscabi_tls_context* context, int required);

Description: Configures whether SNI is required.

Parameters: context: The TLS context handle.; required: Non-zero to require SNI.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_context_set_client_auth

qscabi_status qscabi_tls_context_set_client_auth(qscabi_tls_context* context, int request_client_auth, int require_client_auth);

Description: Configures TLS client-certificate authentication policy.

Parameters: context: The TLS context handle.; request_client_auth: Non-zero to request a client certificate.; require_client_auth: Non-zero to require a valid client certificate.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_context_set_ticket_policy

qscabi_status qscabi_tls_context_set_ticket_policy(qscabi_tls_context* context, const qscabi_tls_ticket_policy* policy);

Description: Sets the TLS session-ticket policy for a context.

Parameters: context: The TLS context handle.; policy: The caller-owned ticket policy to copy into the context.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_context_load_trust_anchor_pem

qscabi_status qscabi_tls_context_load_trust_anchor_pem(qscabi_tls_context* context, const char* pem, size_t pem_length, int self_signed);

Description: Loads a PEM trust anchor into a TLS context.

Parameters: context: The TLS context handle.; pem: The caller-owned PEM certificate buffer.; pem_length: The size of the PEM buffer in bytes.; self_signed: Non-zero if the trust anchor is expected to be self-signed.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_context_load_server_identity_pem

qscabi_status qscabi_tls_context_load_server_identity_pem(qscabi_tls_context* context, const char* chain_pem, size_t chain_pem_length, const char* private_key_pem, size_t private_key_pem_length, qscabi_tls_signature_scheme signature_scheme);

Description: Loads PEM server identity material into a TLS context.

Parameters: context: The TLS context handle.; chain_pem: The caller-owned PEM certificate-chain buffer.; chain_pem_length: The size of the certificate-chain buffer in bytes.; private_key_pem: The caller-owned PEM private-key buffer.; private_key_pem_length: The size of the private-key buffer in bytes.; signature_scheme: The signature scheme associated with the identity key.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_connection_create

qscabi_status qscabi_tls_connection_create(qscabi_tls_connection** connection);

Description: Creates a TLS connection handle.

Parameters: connection: The address that receives the created TLS connection handle.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_connection_destroy

void qscabi_tls_connection_destroy(qscabi_tls_connection* connection);

Description: Destroys a TLS connection handle.

Parameters: connection: The TLS connection handle to destroy, or NULL.

Return: void (no direct status result).

qscabi_tls_client_connect_host

qscabi_status qscabi_tls_client_connect_host(qscabi_tls_connection* connection, const qscabi_tls_context* context, const char* hostname, const char* service);

Description: Connects a TLS client connection to a host and service.

Parameters: connection: The TLS connection handle to initialize.; context: The TLS context containing client policy and trust configuration.; hostname: The null-terminated target hostname.; service: The null-terminated target service or port string.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_connection_send

qscabi_status qscabi_tls_connection_send(qscabi_tls_connection* connection, const uint8_t* input, size_t input_length, size_t* written);

Description: Sends application data on a TLS connection.

Parameters: connection: The TLS connection handle.; input: The caller-owned input buffer.; input_length: The input size in bytes.; written: The optional address that receives the number of bytes written.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_connection_receive

qscabi_status qscabi_tls_connection_receive(qscabi_tls_connection* connection, uint8_t* output, size_t output_length, size_t* read_length);

Description: Receives application data from a TLS connection.

Parameters: connection: The TLS connection handle.; output: The caller-owned output buffer.; output_length: The output buffer size in bytes.; read_length: The optional address that receives the number of bytes read.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_connection_send_frame

qscabi_status qscabi_tls_connection_send_frame(qscabi_tls_connection* connection, const uint8_t* input, size_t input_length);

Description: Sends one length-delimited frame over a TLS connection.

Parameters: connection: The TLS connection handle.; input: The caller-owned frame payload.; input_length: The frame payload size in bytes.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_connection_receive_frame

qscabi_status qscabi_tls_connection_receive_frame(qscabi_tls_connection* connection, uint8_t* output, size_t output_length, size_t* read_length);

Description: Receives one length-delimited frame from a TLS connection.

Parameters: connection: The TLS connection handle.; output: The caller-owned frame-output buffer.; output_length: The output buffer size in bytes.; read_length: The address that receives the received frame size in bytes.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_connection_shutdown

qscabi_status qscabi_tls_connection_shutdown(qscabi_tls_connection* connection);

Description: Performs an orderly TLS connection shutdown.

Parameters: connection: The TLS connection handle.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_connection_cancel

qscabi_status qscabi_tls_connection_cancel(qscabi_tls_connection* connection);

Description: Cancels a TLS connection.

Parameters: connection: The TLS connection handle.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_connection_is_connected

int qscabi_tls_connection_is_connected(const qscabi_tls_connection* connection);

Description: Tests whether a TLS connection is connected.

Parameters: connection: The TLS connection handle.

Return: Non-zero if connected; otherwise zero.

qscabi_tls_connection_is_handshake_complete

int qscabi_tls_connection_is_handshake_complete(const qscabi_tls_connection* connection);

Description: Tests whether the TLS handshake is complete.

Parameters: connection: The TLS connection handle.

Return: Non-zero if the handshake is complete; otherwise zero.

qscabi_tls_connection_get_selected_alpn

qscabi_status qscabi_tls_connection_get_selected_alpn(const qscabi_tls_connection* connection, char* output, size_t output_length, size_t* written);

Description: Gets the selected ALPN protocol from a TLS connection.

Parameters: connection: The TLS connection handle.; output: The caller-owned output buffer for the selected protocol string.; output_length: The output buffer size in bytes.; written: The optional address that receives the number of bytes written.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_connection_get_peer_info

qscabi_status qscabi_tls_connection_get_peer_info(const qscabi_tls_connection* connection, qscabi_tls_peer_info* peer_info);

Description: Gets a peer-information snapshot from a TLS connection.

Parameters: connection: The TLS connection handle.; peer_info: The caller-owned peer-information structure to receive the snapshot.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_listener_create

qscabi_status qscabi_tls_listener_create(qscabi_tls_listener** listener);

Description: Creates a TLS listener handle.

Parameters: listener: The address that receives the created TLS listener handle.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_listener_destroy

void qscabi_tls_listener_destroy(qscabi_tls_listener* listener);

Description: Destroys a TLS listener handle.

Parameters: listener: The TLS listener handle to destroy, or NULL.

Return: void (no direct status result).

qscabi_tls_listener_set_socket_options

qscabi_status qscabi_tls_listener_set_socket_options(qscabi_tls_listener* listener, const qscabi_tls_socket_options* options);

Description: Sets socket options for a TLS listener.

Parameters: listener: The TLS listener handle.; options: The caller-owned socket options to copy into the listener.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_listener_bind

qscabi_status qscabi_tls_listener_bind(qscabi_tls_listener* listener, const qscabi_tls_context* context, const char* address, uint16_t port, qscabi_tls_address_family family);

Description: Binds a TLS listener to an address and port.

Parameters: listener: The TLS listener handle.; context: The TLS context containing server policy and identity configuration.; address: The null-terminated address string to bind.; port: The TCP port to bind.; family: The address-family selector.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_listener_accept

qscabi_status qscabi_tls_listener_accept(qscabi_tls_listener* listener, qscabi_tls_connection* connection);

Description: Accepts a TLS connection from a listener.

Parameters: listener: The TLS listener handle.; connection: The caller-owned TLS connection handle that receives accepted state.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_listener_close

void qscabi_tls_listener_close(qscabi_tls_listener* listener);

Description: Closes a TLS listener.

Parameters: listener: The TLS listener handle to close, or NULL.

Return: void (no direct status result).

qscabi_tls_server_create

qscabi_status qscabi_tls_server_create(qscabi_tls_server** server);

Description: Creates a TLS server handle.

Parameters: server: The address that receives the created TLS server handle.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_server_destroy

void qscabi_tls_server_destroy(qscabi_tls_server* server);

Description: Destroys a TLS server handle.

Parameters: server: The TLS server handle to destroy, or NULL.

Return: void (no direct status result).

qscabi_tls_server_configure

qscabi_status qscabi_tls_server_configure(qscabi_tls_server* server, const qscabi_tls_context* context, const char* address, uint16_t port, qscabi_tls_address_family family);

Description: Configures a TLS server endpoint.

Parameters: server: The TLS server handle.; context: The TLS context containing server policy and identity configuration.; address: The null-terminated address string to bind.; port: The TCP port to bind.; family: The address-family selector.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_server_set_max_clients

qscabi_status qscabi_tls_server_set_max_clients(qscabi_tls_server* server, size_t max_clients);

Description: Sets the maximum number of concurrent TLS server clients.

Parameters: server: The TLS server handle.; max_clients: The maximum number of concurrent clients.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_server_start

qscabi_status qscabi_tls_server_start(qscabi_tls_server* server);

Description: Starts a TLS server.

Parameters: server: The TLS server handle.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_server_start_concurrent

qscabi_status qscabi_tls_server_start_concurrent(qscabi_tls_server* server);

Description: Starts a TLS server using concurrent handling where supported.

Parameters: server: The TLS server handle.

Return: qscabi_status_success on success; otherwise a QSCABI error code.

qscabi_tls_server_stop

void qscabi_tls_server_stop(qscabi_tls_server* server);

Description: Stops a TLS server.

Parameters: server: The TLS server handle to stop, or NULL.

Return: void (no direct status result).

qscabi_tls_context_set_cipher_suites

qscabi_status qscabi_tls_context_set_cipher_suites(qscabi_tls_context* context, const uint16_t* suites, size_t count);

Description: Select an ordered list of supported TLS 1.3 cipher-suite IDs.

Details: Applies atomically after validating the entire list. No network handshake occurs.

Parameters: context: Borrowed initialized TLS context; remains caller-owned.; suites: Borrowed array of 16-bit wire IDs, not retained.; count: Number of IDs (1..QSC TLS maximum); duplicates and unknown IDs rejected.

Return: Success when the full list is installed; invalid_argument otherwise.

qscabi_tls_context_set_named_groups

qscabi_status qscabi_tls_context_set_named_groups(qscabi_tls_context* context, const uint16_t* groups, size_t count);

Description: Select an ordered list of TLS classical, PQC, or hybrid group IDs.

Parameters: context: Borrowed TLS context.; groups: Borrowed array of TLS named-group wire IDs.; count: Number of IDs; must be between one and backend maximum.

Return: Success or mapped native error; rejects repeated/unknown groups.

qscabi_tls_context_set_signature_ids

qscabi_status qscabi_tls_context_set_signature_ids(qscabi_tls_context* context, const uint16_t* schemes, size_t count);

Description: Select an ordered list of TLS CertificateVerify signature IDs.

Parameters: context: Borrowed TLS context.; schemes: Borrowed 16-bit signature IDs, not retained.; count: Number of IDs; nonzero and within the backend limit.

Return: Success or a QSCABI validation/TLS status code.

qscabi_tls_context_load_trust_anchor_file

qscabi_status qscabi_tls_context_load_trust_anchor_file(qscabi_tls_context* context, const char* path, int self_signed);

Description: Load a trust-anchor certificate from a filesystem path.

Parameters: context: Borrowed initialized context.; path: NUL-terminated certificate file path read immediately, not retained.; self_signed: Nonzero to require a self-signed trust anchor.

Return: Native TLS socket status mapped to QSCABI; I/O errors are failures.

qscabi_tls_context_load_trust_anchor_bundle_file

qscabi_status qscabi_tls_context_load_trust_anchor_bundle_file(qscabi_tls_context* context, const char* path, int self_signed);

Description: Load a trust-anchor bundle from a file.

Parameters: context: Borrowed initialized context.; path: Borrowed null-terminated PEM/DER bundle path.; self_signed: Nonzero to request self-signed trust anchors.

Return: Mapped status, without preserving caller pointers.

qscabi_tls_context_load_crl_file

qscabi_status qscabi_tls_context_load_crl_file(qscabi_tls_context* context, const char* path);

Description: Load a certificate revocation list from disk.

Parameters: context: Borrowed initialized context.; path: Borrowed null-terminated CRL file path.

Return: Success only after native loader accepts the CRL.

qscabi_tls_context_load_server_identity_files

qscabi_status qscabi_tls_context_load_server_identity_files(qscabi_tls_context* context, const char* chain_path, const char* key_path, qscabi_tls_signature_scheme scheme);

Description: Load a server certificate chain and private key from files.

Parameters: context: Borrowed initialized context.; chain_path: Borrowed certificate-chain file path.; key_path: Borrowed private-key file path; no key bytes cross the ABI.; scheme: Signature scheme used for CertificateVerify.

Return: Success or mapped unsupported/credential/TLS error.

qscabi_tls_context_load_client_identity_files

qscabi_status qscabi_tls_context_load_client_identity_files(qscabi_tls_context* context, const char* chain_path, const char* key_path, qscabi_tls_signature_scheme scheme);

Description: Load a client certificate chain and private key for mTLS.

Parameters: context: Borrowed initialized context.; chain_path: Borrowed certificate-chain file path.; key_path: Borrowed private-key file path.; scheme: Signature scheme used in the client CertificateVerify.

Return: Success or mapped TLS certificate/private-key error.

qscabi_tls_context_add_server_identity_files

qscabi_status qscabi_tls_context_add_server_identity_files(qscabi_tls_context* context, const char* hostname, const char* chain_path, const char* key_path, qscabi_tls_signature_scheme scheme);

Description: Add an SNI-selectable server identity from two files.

Parameters: context: Borrowed initialized context.; hostname: Borrowed DNS hostname/pattern; NUL-terminated, not retained.; chain_path: Borrowed certificate-chain file path.; key_path: Borrowed private-key file path.; scheme: Signature scheme; unsupported choices fail before filesystem access.

Return: Success or a validation/credential/unsupported error.

23.18 qscabi_version.h — 12 functions

ABI versions, build metadata and capabilities.

qscabi_version_major

uint32_t qscabi_version_major(void);

Description: Gets the QSCABI major version number.

Return: Returns the QSCABI major version number.

Notes: The major version is incremented when the public ABI contains incompatible changes that require wrapper or application review.

qscabi_version_minor

uint32_t qscabi_version_minor(void);

Description: Gets the QSCABI minor version number.

Return: Returns the QSCABI minor version number.

Notes: The minor version is incremented when new ABI-compatible functionality is added to the bridge library.

qscabi_version_patch

uint32_t qscabi_version_patch(void);

Description: Gets the QSCABI patch version number.

Return: Returns the QSCABI patch version number.

Notes: The patch version is incremented for ABI-compatible corrections, maintenance updates, and implementation fixes.

qscabi_version_string

const char* qscabi_version_string(void);

Description: Gets the complete QSCABI version string.

Return: Returns a null-terminated QSCABI version string.

Notes: The returned pointer references a static string owned by QSCABI. The caller must not modify or free the returned pointer.

qscabi_qsc_version_string

const char* qscabi_qsc_version_string(void);

Description: Gets the linked QSC implementation version string.

Return: Returns a null-terminated QSC implementation version string.

Notes: This function reports the QSC version used as the implementation source for the loaded QSCABI library. The returned pointer references a static string owned by QSCABI.

qscabi_build_platform

const char* qscabi_build_platform(void);

Description: Gets the platform identifier for the QSCABI build.

Return: Returns a null-terminated platform identifier string.

Notes: The returned string identifies the operating-system or platform family targeted by the compiled QSCABI library.

qscabi_build_compiler

const char* qscabi_build_compiler(void);

Description: Gets the compiler identifier for the QSCABI build.

Return: Returns a null-terminated compiler identifier string.

Notes: The returned string identifies the compiler family used to build the loaded QSCABI library, such as MSVC, GCC, or Clang.

qscabi_abi_is_compatible

uint32_t qscabi_abi_is_compatible(uint32_t major, uint32_t minor);

Description: Check ABI major and minimum required minor version.

Parameters: major: Expected major version; must match the loaded QSCABI library.; minor: Minimum supported minor version of the same ABI major.

Return: One on compatibility; zero otherwise. No allocation or I/O.

qscabi_capability_flags

uint64_t qscabi_capability_flags(void);

Description: Get compiled feature capabilities, not hardware-presence evidence.

Return: Bitmask of QSCABI_CAPABILITY_* flags, determined at build time.

qscabi_build_backend

const char* qscabi_build_backend(void);

Description: Get backend description for native-build and packaging diagnostics.

Return: Library-owned null-terminated immutable static string.

qscabi_qsc_build_profile

const char* qscabi_qsc_build_profile(void);

Description: Get the native QSC source-profile description.

Return: Library-owned immutable string; not a runtime ISA dispatcher result.

qscabi_enabled_features

const char* qscabi_enabled_features(void);

Description: Get the library-owned summary of features compiled into this ABI.

Details: The string is diagnostic and does not guarantee CPU ISA support, physical device presence, independent cryptographic approval or negotiated TLS interoperability. Do not free or modify the returned pointer.

Return: Immutable NUL-terminated feature string, owned by QSCABI.

23.19 qscabi_x509.h — 39 functions

Certificate, chain, store, CSR, issuance and CRL functionality.

qscabi_x509_certificate_create

qscabi_status qscabi_x509_certificate_create(qscabi_x509_certificate** certificate);

Description: Allocates an opaque X.509 certificate handle.

Parameters: certificate: Receives the newly allocated certificate handle.

Return: Returns qscabi_status_success on success; otherwise returns an error status such as qscabi_status_invalid_argument or qscabi_status_allocation_failure.

qscabi_x509_certificate_destroy

void qscabi_x509_certificate_destroy(qscabi_x509_certificate* certificate);

Description: Releases an opaque X.509 certificate handle.

Parameters: certificate: The certificate handle to release, or null.

Return: void (no direct status result).

qscabi_x509_certificate_load_memory

qscabi_status qscabi_x509_certificate_load_memory(qscabi_x509_certificate* certificate, const uint8_t* data, size_t data_length);

Description: Loads an X.509 certificate from a memory buffer.

Details: The input buffer may contain a supported certificate encoding. The buffer is caller-owned and is not retained after the function returns.

Parameters: certificate: The certificate handle to populate.; data: The caller-owned certificate buffer.; data_length: The size of data in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status value.

qscabi_x509_certificate_load_pem

qscabi_status qscabi_x509_certificate_load_pem(qscabi_x509_certificate* certificate, const char* pem, size_t pem_length);

Description: Loads an X.509 certificate from PEM text.

Parameters: certificate: The certificate handle to populate.; pem: The caller-owned PEM text buffer.; pem_length: The size of pem in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status value.

qscabi_x509_certificate_load_der

qscabi_status qscabi_x509_certificate_load_der(qscabi_x509_certificate* certificate, const uint8_t* der, size_t der_length);

Description: Loads an X.509 certificate from DER bytes.

Parameters: certificate: The certificate handle to populate.; der: The caller-owned DER buffer.; der_length: The size of der in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status value.

qscabi_x509_certificate_export_der

qscabi_status qscabi_x509_certificate_export_der(const qscabi_x509_certificate* certificate, uint8_t* output, size_t output_length, size_t* written);

Description: Exports a loaded certificate as DER.

Parameters: certificate: The loaded certificate handle.; output: The caller-owned output buffer.; output_length: The size of output in bytes.; written: Receives the number of bytes written, or the required size when available.

Return: Returns qscabi_status_success on success, qscabi_status_buffer_too_small if output is too small, or another QSCABI status value on failure.

qscabi_x509_certificate_export_pem

qscabi_status qscabi_x509_certificate_export_pem(const qscabi_x509_certificate* certificate, char* output, size_t output_length, size_t* written);

Description: Exports a loaded certificate as PEM text.

Parameters: certificate: The loaded certificate handle.; output: The caller-owned character output buffer.; output_length: The size of output in bytes.; written: Receives the number of bytes written, or the required size when available.

Return: Returns qscabi_status_success on success, qscabi_status_buffer_too_small if output is too small, or another QSCABI status value on failure.

qscabi_x509_certificate_subject_string

qscabi_status qscabi_x509_certificate_subject_string(const qscabi_x509_certificate* certificate, char* output, size_t output_length, size_t* written);

Description: Gets the certificate subject as a formatted string.

Parameters: certificate: The loaded certificate handle.; output: The caller-owned character output buffer.; output_length: The size of output in bytes.; written: Receives the number of bytes written, or the required size when available.

Return: Returns qscabi_status_success on success, qscabi_status_buffer_too_small if output is too small, or another QSCABI status value on failure.

qscabi_x509_certificate_issuer_string

qscabi_status qscabi_x509_certificate_issuer_string(const qscabi_x509_certificate* certificate, char* output, size_t output_length, size_t* written);

Description: Gets the certificate issuer as a formatted string.

Parameters: certificate: The loaded certificate handle.; output: The caller-owned character output buffer.; output_length: The size of output in bytes.; written: Receives the number of bytes written, or the required size when available.

Return: Returns qscabi_status_success on success, qscabi_status_buffer_too_small if output is too small, or another QSCABI status value on failure.

qscabi_x509_certificate_subject_common_name

qscabi_status qscabi_x509_certificate_subject_common_name(const qscabi_x509_certificate* certificate, char* output, size_t output_length, size_t* written);

Description: Gets the certificate subject common name.

Parameters: certificate: The loaded certificate handle.; output: The caller-owned character output buffer.; output_length: The size of output in bytes.; written: Receives the number of bytes written, or the required size when available.

Return: Returns qscabi_status_success on success, qscabi_status_buffer_too_small if output is too small, or another QSCABI status value on failure.

qscabi_x509_certificate_subject_dns_name_count

size_t qscabi_x509_certificate_subject_dns_name_count(const qscabi_x509_certificate* certificate);

Description: Gets the number of DNS subject-alternative-name entries.

Parameters: certificate: The loaded certificate handle.

Return: Returns the DNS SAN entry count. A return value of zero can indicate no DNS SAN entries or an invalid/unloaded certificate handle.

qscabi_x509_certificate_subject_dns_name

qscabi_status qscabi_x509_certificate_subject_dns_name(const qscabi_x509_certificate* certificate, size_t index, char* output, size_t output_length, size_t* written);

Description: Gets a DNS subject-alternative-name entry by index.

Parameters: certificate: The loaded certificate handle.; index: The zero-based DNS SAN index to retrieve.; output: The caller-owned character output buffer.; output_length: The size of output in bytes.; written: Receives the number of bytes written, or the required size when available.

Return: Returns qscabi_status_success on success, qscabi_status_not_found if index is not present, qscabi_status_buffer_too_small if output is too small, or another QSCABI status value on failure.

qscabi_x509_certificate_match_hostname

qscabi_status qscabi_x509_certificate_match_hostname(const qscabi_x509_certificate* certificate, const char* hostname);

Description: Checks whether a certificate matches a host name.

Parameters: certificate: The loaded certificate handle.; hostname: The expected host name as a null-terminated string.

Return: Returns qscabi_status_success when the certificate matches the host name; otherwise returns a verification or validation status.

qscabi_x509_certificate_check_role

qscabi_status qscabi_x509_certificate_check_role(const qscabi_x509_certificate* certificate, qscabi_x509_certificate_role role, const char* hostname);

Description: Checks whether a certificate satisfies a requested role.

Parameters: certificate: The loaded certificate handle.; role: The expected certificate role.; hostname: The expected host name for roles that require host-name validation, or null when not required.

Return: Returns qscabi_status_success when the certificate satisfies the requested role; otherwise returns a verification or validation status.

qscabi_x509_chain_create

qscabi_status qscabi_x509_chain_create(qscabi_x509_chain** chain, size_t certificate_capacity);

Description: Allocates an opaque X.509 certificate-chain handle.

Parameters: chain: Receives the newly allocated chain handle.; certificate_capacity: The maximum number of certificates the chain should support.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status value.

qscabi_x509_chain_destroy

void qscabi_x509_chain_destroy(qscabi_x509_chain* chain);

Description: Releases an opaque X.509 certificate-chain handle.

Parameters: chain: The chain handle to release, or null.

Return: void (no direct status result).

qscabi_x509_chain_load_memory

qscabi_status qscabi_x509_chain_load_memory(qscabi_x509_chain* chain, const uint8_t* data, size_t data_length);

Description: Loads a certificate chain from a memory buffer.

Details: The input buffer may contain a supported chain encoding. The buffer is caller-owned and is not retained after the function returns.

Parameters: chain: The chain handle to populate.; data: The caller-owned chain buffer.; data_length: The size of data in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status value.

qscabi_x509_chain_count

size_t qscabi_x509_chain_count(const qscabi_x509_chain* chain);

Description: Gets the number of certificates currently loaded into a chain.

Parameters: chain: The certificate-chain handle.

Return: Returns the number of certificates in the chain. A return value of zero can indicate an empty chain or an invalid chain handle.

qscabi_x509_store_create

qscabi_status qscabi_x509_store_create(qscabi_x509_store** store, size_t anchor_capacity);

Description: Allocates an opaque X.509 trust-store handle.

Parameters: store: Receives the newly allocated trust-store handle.; anchor_capacity: The maximum number of trust anchors the store should support.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status value.

qscabi_x509_store_destroy

void qscabi_x509_store_destroy(qscabi_x509_store* store);

Description: Releases an opaque X.509 trust-store handle.

Parameters: store: The trust-store handle to release, or null.

Return: void (no direct status result).

qscabi_x509_store_add_anchor_memory

qscabi_status qscabi_x509_store_add_anchor_memory(qscabi_x509_store* store, const uint8_t* data, size_t data_length, int self_signed);

Description: Adds a trust anchor to a trust store from memory.

Parameters: store: The trust-store handle to update.; data: The caller-owned trust-anchor certificate buffer.; data_length: The size of data in bytes.; self_signed: Non-zero if the trust anchor is expected to be self-signed; zero otherwise.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status value.

qscabi_x509_store_count

size_t qscabi_x509_store_count(const qscabi_x509_store* store);

Description: Gets the number of trust anchors currently loaded into a store.

Parameters: store: The trust-store handle.

Return: Returns the number of trust anchors in the store. A return value of zero can indicate an empty store or an invalid store handle.

qscabi_x509_verify_peer_certificates

qscabi_status qscabi_x509_verify_peer_certificates(const qscabi_x509_chain* chain, const qscabi_x509_store* store, qscabi_x509_profile_preset preset, const char* hostname);

Description: Verifies a peer certificate chain against a trust store.

Parameters: chain: The peer certificate-chain handle.; store: The trust-store handle containing trusted anchors.; preset: The verification profile preset.; hostname: The expected peer host name for profiles that require host-name validation, or null when not required.

Return: Returns qscabi_status_success when verification succeeds; otherwise returns a verification, X.509, or validation status.

qscabi_x509_status_string

const char* qscabi_x509_status_string(qscabi_status status);

Description: Converts a QSCABI X.509-related status value to a static string.

Parameters: status: The status value to describe.

Return: Returns a static null-terminated string. The caller must not modify or free the returned pointer.

qscabi_x509_csr_create

qscabi_status qscabi_x509_csr_create(qscabi_x509_csr** csr);

Description: Allocate an empty native certificate signing request handle.

Parameters: csr: Out pointer set to an owned handle on success, otherwise NULL.

Return: Success or invalid_argument/allocation_failure.

qscabi_x509_csr_destroy

void qscabi_x509_csr_destroy(qscabi_x509_csr* csr);

Description: Release a CSR handle and any decoded DER memory.

Parameters: csr: Previously allocated handle or NULL; safe as a no-op for NULL.

Return: void (no direct status result).

qscabi_x509_csr_init_p256

qscabi_status qscabi_x509_csr_init_p256(qscabi_x509_csr* csr, const char* common_name, const uint8_t* public_key, size_t public_key_length);

Description: Initialize a CSR with a P-256 subject name and uncompressed public key.

Parameters: csr: Mutable CSR handle; existing state is cleared.; common_name: Borrowed nonempty, NUL-terminated UTF-8 common name.; public_key: Borrowed 64-byte X||Y P-256 public key, not retained.; public_key_length: Must be exactly 64 bytes.

Return: Success or invalid_argument/unsupported/native X.509 failure.

qscabi_x509_csr_add_dns

qscabi_status qscabi_x509_csr_add_dns(qscabi_x509_csr* csr, const char* dns);

Description: Add a DNS SAN to a mutable CSR's extension request.

Parameters: csr: Mutable initialized CSR handle.; dns: Borrowed, nonempty, NUL-terminated DNS name (not retained).

Return: Success or invalid_argument/native X.509 status.

qscabi_x509_csr_sign_der

qscabi_status qscabi_x509_csr_sign_der(const qscabi_x509_csr* csr, qscabi_x509_sign_callback signer, void* context, uint8_t* output, size_t* output_length);

Description: Sign a CSR through a caller-owned signing callback.

Parameters: csr: Initialized CSR handle, borrowed for the call.; signer: Nonthrowing external ECDSA-with-SHA256 callback.; context: Borrowed signing-key context, never retained.; output: Caller-owned DER output buffer.; output_length: In/out capacity and written byte count.

Return: Success, buffer_too_small or X.509/signature failure.

qscabi_x509_csr_load

qscabi_status qscabi_x509_csr_load(qscabi_x509_csr* csr, const uint8_t* data, size_t length);

Description: Decode an externally supplied DER CSR into an opaque handle.

Parameters: csr: Mutable CSR handle; previous decoded state is cleared.; data: Borrowed DER bytes, copied/owned by the native parser.; length: Byte count of DER data.

Return: Success or malformed-input/X.509 error.

qscabi_x509_csr_verify

qscabi_status qscabi_x509_csr_verify(const qscabi_x509_csr* csr);

Description: Cryptographically verify the CSR signature against its embedded SPKI.

Details: Verification does not establish trust in the requester or domain.

Parameters: csr: Borrowed loaded CSR handle.

Return: Success on valid signature; verification_failure otherwise.

qscabi_x509_issue_tls_server_p256

qscabi_status qscabi_x509_issue_tls_server_p256(const qscabi_x509_csr* csr, const qscabi_x509_certificate* issuer, const uint8_t* serial, size_t serial_length, const qscabi_x509_time* not_before, const qscabi_x509_time* not_after, qscabi_x509_sign_callback signer, void* context, uint8_t* output, size_t* output_length);

Description: Issue a P-256 TLS server certificate from a signed CSR and trusted CA.

Parameters: csr: Signed CSR containing a valid P-256 public key; borrowed.; issuer: Loaded issuer certificate handle; borrowed.; serial: Borrowed nonzero serial octets, positive DER INTEGER semantics.; serial_length: Serial length, bounded by the native X.509 limit.; not_before: Inclusive certificate start UTC time, borrowed.; not_after: Exclusive certificate expiration UTC time, borrowed.; signer: External issuer-private-key callback; private material not retained.; context: Borrowed signing-key context.; output: Caller-owned DER certificate destination.; output_length: In/out capacity and produced length.

Return: Success or input/policy/X.509/signing error. Issuance is not trust.

qscabi_x509_crl_builder_create

qscabi_status qscabi_x509_crl_builder_create(qscabi_x509_crl_builder** builder, const qscabi_x509_certificate* issuer, const qscabi_x509_time* this_update, const qscabi_x509_time* next_update);

Description: Create an issuer-bound CRL builder with validated update times.

Parameters: builder: Out pointer set to an owned builder, otherwise NULL.; issuer: Loaded CA certificate with CRL-signing authority, borrowed.; this_update: UTC thisUpdate date/time, borrowed.; next_update: UTC nextUpdate date/time, must follow thisUpdate.

Return: Success or invalid_argument/allocation/X.509 failure.

qscabi_x509_crl_builder_destroy

void qscabi_x509_crl_builder_destroy(qscabi_x509_crl_builder* builder);

Description: Release a CRL builder and zero its native mutable structure.

Parameters: builder: Owned handle or NULL; invalidated after return.

Return: void (no direct status result).

qscabi_x509_crl_set_number

qscabi_status qscabi_x509_crl_set_number(qscabi_x509_crl_builder* builder, const uint8_t* value, size_t length);

Description: Add a positive CRLNumber extension to a mutable builder.

Parameters: builder: Mutable CRL builder.; value: Borrowed unsigned big-endian CRLNumber bytes, not retained.; length: Length of value, within native extension limits.

Return: Success or validation/X.509 error.

qscabi_x509_crl_add_revoked

qscabi_status qscabi_x509_crl_add_revoked(qscabi_x509_crl_builder* builder, const uint8_t* serial, size_t serial_length, const qscabi_x509_time* revoked_at);

Description: Append a revoked certificate serial and UTC revocation date.

Parameters: builder: Mutable issuer-bound CRL builder.; serial: Borrowed positive serial bytes, not retained.; serial_length: Number of serial bytes.; revoked_at: Valid UTC date/time, borrowed.

Return: Success, or invalid_argument if duplicate/invalid.

qscabi_x509_crl_sign_der

qscabi_status qscabi_x509_crl_sign_der(const qscabi_x509_crl_builder* builder, const qscabi_x509_certificate* issuer, qscabi_x509_sign_callback signer, void* context, uint8_t* output, size_t* output_length);

Description: Produce signed DER CRL using an external issuer private key.

Parameters: builder: Initialized CRL builder, borrowed.; issuer: Loaded issuer certificate (must match builder).; signer: External nonthrowing P-256 SHA256 callback.; context: Borrowed signing context.; output: Caller-owned DER buffer.; output_length: In/out capacity and written byte count.

Return: Success or native validation/signature/size error.

qscabi_x509_crl_verify_der

qscabi_status qscabi_x509_crl_verify_der(const uint8_t* der, size_t length, const qscabi_x509_certificate* issuer, const qscabi_x509_time* verification_time);

Description: Verify DER CRL issuer, validity interval, and cryptographic signature.

Parameters: der: Borrowed DER CRL octets.; length: Number of DER octets.; issuer: Loaded trusted issuer certificate, borrowed.; verification_time: Caller-provided UTC time for revocation validation.

Return: Success only for a current and correctly signed CRL.

qscabi_x509_store_add_crl_memory

qscabi_status qscabi_x509_store_add_crl_memory(qscabi_x509_store* store, const uint8_t* der, size_t length);

Description: Add a DER CRL to an existing QSCABI X.509 trust store.

Parameters: store: Mutable initialized trust store, borrowed.; der: Borrowed DER CRL bytes, not retained after native parsing.; length: Number of bytes supplied.

Return: Success or validation/CRL/store-capacity error.

23.20 qscabi_xof.h — 3 functions

One-shot SHAKE XOF functions.

qscabi_shake_128_compute

qscabi_status qscabi_shake_128_compute(uint8_t* output, size_t output_length, const uint8_t* input, size_t input_length);

Description: Generates SHAKE-128 extendable output from an input message.

Parameters: output: [uint8_t*] Caller-owned output buffer that receives SHAKE output.; output_length: [size_t] Number of output bytes requested; must be greater than zero.; input: [const uint8_t*] Caller-owned input message buffer.; input_length: [size_t] Length of input in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

qscabi_shake_256_compute

qscabi_status qscabi_shake_256_compute(uint8_t* output, size_t output_length, const uint8_t* input, size_t input_length);

Description: Generates SHAKE-256 extendable output from an input message.

Parameters: output: [uint8_t*] Caller-owned output buffer that receives SHAKE output.; output_length: [size_t] Number of output bytes requested; must be greater than zero.; input: [const uint8_t*] Caller-owned input message buffer.; input_length: [size_t] Length of input in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

qscabi_shake_512_compute

qscabi_status qscabi_shake_512_compute(uint8_t* output, size_t output_length, const uint8_t* input, size_t input_length);

Description: Generates SHAKE-512 extendable output from an input message.

Parameters: output: [uint8_t*] Caller-owned output buffer that receives SHAKE output.; output_length: [size_t] Number of output bytes requested; must be greater than zero.; input: [const uint8_t*] Caller-owned input message buffer.; input_length: [size_t] Length of input in bytes.

Return: Returns qscabi_status_success on success; otherwise returns a QSCABI status code.

Annex A: Public ABI Data Types and Constants

A.1 Opaque objects and runtime selector families

Family	Source header	Public contract
Certificate, chain and trust store	qscabi_x509.h	Handles allocated and freed only through matching create/destroy functions.
CSR and CRL builder	qscabi_x509.h	Stateful signing / verification objects, including application callback ownership.
TLS context, connection, listener, server	qscabi_tls.h	Opaque native transport/protocol state; no QSC socket structures exposed.
Digest, XOF, MAC and CTR stream state	qscabi_stream.h	Create/update/finalize/destroy with strict lifecycle and single-thread ownership.
Hardware provider and key	qscabi_hardware.h	Hardware-callback state and key handles; backend object must outlive open provider.
Explicit KEM/curve/signature/SLH-DSA	qscabi_explicit_*.h	Stable enum selectors with size metadata, reject unsupported variants.

Table A.1 Principal handle classes

A.2 Version and capability constants

Constant	Header-defined value	Meaning
QSCABI_VERSION_MAJOR	0U	Public compiled version metadata
QSCABI_VERSION_MINOR	10U	Public compiled version metadata
QSCABI_VERSION_PATCH	0U	Public compiled version metadata
QSCABI_VERSION_STRING	"0.10.0-dev"	Public compiled version metadata
QSCABI_CAPABILITY_EXPLICIT_ALGORITHMS	UINT64_C(0x0000000000001000)	Compiled capability bit; does not certify runtime support
QSCABI_CAPABILITY_STREAMING	UINT64_C(0x0000000000002000)	Compiled capability bit; does not certify runtime support
QSCABI_CAPABILITY_X509_ISSUANCE	UINT64_C(0x0000000000004000)	Compiled capability bit; does not certify runtime support
QSCABI_CAPABILITY_TLS_PREFERENCES	UINT64_C(0x0000000000008000)	Compiled capability bit; does not certify runtime support
QSCABI_CAPABILITY_HARDWARE_PROVIDER	UINT64_C(0x0000000000010000)	Compiled capability bit; does not certify runtime support

Table A.2 Version and capability selectors

Annex B: Source and Test Index

Public header	Functions	Module
qscabi_asymmetric.h	36	asymmetric
qscabi_encoding.h	12	encoding
qscabi_entropy.h	10	entropy
qscabi_explicit_curve.h	5	explicit curve
qscabi_explicit_kem.h	4	explicit kem
qscabi_explicit_signature.h	4	explicit signature
qscabi_explicit_slhdsa.h	4	explicit slhdsa
qscabi_hardware.h	14	hardware
qscabi_hash.h	12	hash
qscabi_kdf.h	12	kdf
qscabi_mac.h	23	mac
qscabi_memory.h	2	memory
qscabi_pqc.h	42	pqc
qscabi_status.h	1	status
qscabi_stream.h	18	stream
qscabi_symmetric.h	27	symmetric
qscabi_tls.h	49	tls
qscabi_version.h	12	version
qscabi_x509.h	39	x509
qscabi_xof.h	3	xof

Table B.1 Public function inventory by header

Module source .c files are in QSCABI/; corresponding named test files are in QSCABITest/. The umbrella qscabi.h, common linkage header and validation helper header intentionally contain no QSCABI_API function declarations. The user-supplied ZIP includes seven C examples and a multi-language ABI smoke-test directory. Existing compiled object files and logs are historical build artifacts, not independently reproduced evidence.

Annex C: Implementation Checklist

- Pin the precise QSCABI and external QSC revisions, FULL profile, platform, toolchain and ISA-compatible binary.
- Verify ABI major/minor and capability flags before using optional functions.
- Audit every public function used by the application against the actual installed header signature.
- Represent size_t, fixed-width integers, enums, pointer-to-pointer outputs and status codes correctly in every binding.
- Allocate caller-owned buffers and distinguish capacity from actual bytes written.
- Treat all secret material as sensitive; clear private keys, seeds and derived keys after use.
- Use the exact create/destroy path for each opaque handle; never free from a foreign allocator.
- Reject AEAD overlap and tag-verification failures; do not release tentative plaintext.
- Check explicit PQC/curve/signature/SLH-DSA selector support and use returned sizes.
- Authenticate peer identities, TLS certificate paths and trust-anchor policies independently of parse status.
- Use supported file-based TLS credential loaders; do not rely on unsupported memory-loader stubs.
- Qualify native and managed runtimes, full cryptographic negative cases, actual TLS peers and physical devices where required.
- Audit 329 public exports, absence of qsc_* native leaks, distribution license, and delivered binary hashes.
- Record actual PASS/FAIL with execution environment; do not promote historical project logs to an unexecuted current certification.

A completed API inventory and tested release matrix are required before approving this development-stage ABI for commercial redistribution. The document does not constitute a formal security certification.